

# Programación estructurada

Joan Arnedo Moreno

Programación básica (ASX)  
Programación (DAM)  
Programación (DAW)



## Índice

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| Introducción                                                                            | 5  |
| Resultados de aprendizaje                                                               | 7  |
| 1 Algoritmos                                                                            | 9  |
| 1.1 Definición del problema . . . . .                                                   | 9  |
| 1.2 Diseño del algoritmo . . . . .                                                      | 10 |
| 1.2.1 Programación estructurada . . . . .                                               | 11 |
| 1.2.2 Representación de algoritmos . . . . .                                            | 13 |
| 1.3 Implementación del programa . . . . .                                               | 14 |
| 1.4 Verificación y pruebas . . . . .                                                    | 14 |
| 1.5 Puesta en marcha y mantenimiento . . . . .                                          | 16 |
| 2 Estructuras de selección                                                              | 17 |
| 2.1 Una desviación temporal del camino: selección simple . . . . .                      | 17 |
| 2.1.1 Sintaxis y comportamiento . . . . .                                               | 18 |
| 2.1.2 Ejemplo: calcular un descuento . . . . .                                          | 19 |
| 2.1.3 Aspectos importantes de la selección simple . . . . .                             | 21 |
| 2.2 Dos caminos alternativos: la sentencia "if/else" . . . . .                          | 23 |
| 2.2.1 Sintaxis y comportamiento . . . . .                                               | 23 |
| 2.2.2 Ejemplo: adivina el número secreto . . . . .                                      | 24 |
| 2.3 Varios caminos: la sentencia "if/else if/else" . . . . .                            | 26 |
| 2.3.1 Sintaxis y comportamiento . . . . .                                               | 26 |
| 2.3.2 Ejemplo: transformar evaluación numérica a texto . . . . .                        | 27 |
| 2.4 Combinación de estructuras de selección . . . . .                                   | 31 |
| 2.4.1 Ejemplo: descuento máximo y control de errores . . . . .                          | 31 |
| 2.4.2 Múltiples bloques de instrucciones y ámbito de variables . . . . .                | 33 |
| 2.5 El conmutador: la sentencia "switch" . . . . .                                      | 35 |
| 2.5.1 Sintaxis y comportamiento . . . . .                                               | 36 |
| 2.5.2 Ejemplo simple: diferentes opciones en un menú . . . . .                          | 37 |
| 2.5.3 Ejemplo con propagación de instrucciones: los días de un mes . . . . .            | 39 |
| 2.6 Control de errores en la entrada básica mediante estructuras de selección . . . . . | 42 |
| 2.7 Soluciones de los retos propuestos . . . . .                                        | 44 |
| 3 Estructuras de repetición                                                             | 47 |
| 3.1 Control de las estructuras repetitivas . . . . .                                    | 48 |
| 3.2 Repetir si se cumple una condición: la sentencia while . . . . .                    | 49 |
| 3.2.1 Sintaxis y estructura . . . . .                                                   | 49 |
| 3.2.2 Ejemplo: ahorrarse de escribir lo mismo muchas veces . . . . .                    | 50 |
| 3.2.3 Ejemplo: aprovechar un contador . . . . .                                         | 52 |
| 3.2.4 Ejemplo: no siempre se suma u . . . . .                                           | 53 |
| 3.2.5 Ejemplo: acumular cálculos . . . . .                                              | 55 |
| 3.2.6 Ejemplo: semáforos . . . . .                                                      | 57 |
| 3.2.7 Ejemplo: semáforos y contadores a la vez . . . . .                                | 59 |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 3.3 Repetir al menos una vez: la sentencia "do/while" . . . . .     | 61 |
| 3.3.1 Sintaxis y estructura . . . . .                               | 62 |
| 3.3.2 Ejemplo: control de entrada por teclado . . . . .             | 63 |
| 3.4 Repetir un cierto número de veces: la sentencia "for" . . . . . | 64 |
| 3.4.1 Sintaxis y estructura . . . . .                               | 65 |
| 3.4.2 Ejemplo: la tabla de multiplicar, versión 2 . . . . .         | 66 |
| 3.4.3 Ejemplo: más allá de sumar y restar . . . . .                 | 67 |
| 3.5 Combinación de estructuras de selección . . . . .               | 68 |
| 3.5.1 Ejemplo: la tabla de multiplicar, versión 3 . . . . .         | 68 |
| 3.5.2 Ejemplo: adivinar el número secreto, versión 3 . . . . .      | 69 |
| 3.6 Soluciones de los retos propuestos . . . . .                    | 71 |

## Introducción

Si bien cuando se habla de las tareas del programador lo primero que viene a la cabeza es la generación del código fuente, en realidad su trabajo va más allá y engloba un proceso más general que va desde saber definir exactamente el problema a resolver hasta el despliegue de la aplicación, pasando por garantizar que el programa es correcto. Por tanto, antes de proseguir con los aspectos vinculados a la generación de código fuente, vale la pena dar una visión general del ciclo de vida de una aplicación.

Dentro de este ciclo de vida es especialmente importante el hecho de que, sin siquiera abrir sus herramientas de programación, debe reflexionar y pensar cómo será su programa: esquematizar cuál es la serie de pasos que deben seguirse para que haga correctamente su tarea. Es decir, definir algoritmo.

Para definir el algoritmo, es necesario tener presente que un programa se dedica principalmente a transformar y manipular datos. Por tanto, todas las instrucciones vinculadas a su gestión son de gran importancia. En este aspecto, la tarea del programador es establecer cómo utilizar variables en la evaluación de expresiones y el almacenamiento del resultado. Si un programa se ciñe exclusivamente a ello, su estructura siempre tendrá forma de secuencia de instrucciones que se van ejecutando de forma consecutiva, una tras otra. Una vez que todas y cada una de las instrucciones se han ejecutado y se llega a la última de todas, el programa se da por finalizado. Todas las instrucciones siempre se terminan ejecutando, y cada una lo hará una sola vez.

Ahora bien, si se fija en los programas que usa habitualmente, éstos están llenos de situaciones en las que esto no se cumple. En un videojuego, cuando se inicia, a menudo existen diferentes opciones: empezar la partida, recuperar una partida guardada, entrar en la configuración, etc. Además, cuando se acaba la partida, nunca ocurre que se acaba la ejecución del programa y para volver a jugar hay que volver a ejecutarlo. Normalmente, le pregunta si desea jugar de nuevo. Por tanto, está claro que dentro de un programa hay muchas instrucciones, pero ni todas se ejecutan siempre, ni el camino que sigue la ejecución es único cada vez que lo ponemos en marcha. Éste sigue una estructura más compleja, en la que hay bifurcaciones y bucles. Y es que, precisamente, otra tarea muy importante del programador será establecer esta estructura: cuál es el flujo de control, o de ejecución, del programa, cada vez que se ponga en marcha. Por ello, deberá considerar bajo qué condiciones es necesario ejecutar o no ciertas instrucciones, y cuando es necesario que algunas se ejecuten repetidas veces. En esta unidad verá cómo se hace, mediante las sentencias que permiten declarar estructuras de control.

Como paso previo a estudiar las estructuras de control, el apartado "Ciclo de vida de una aplicación" resume brevemente cuáles son las tareas del programador a la hora de crear un programa y cómo puede diseñar sus algoritmos.

En el apartado "Estructuras de selección" se explican qué sentencias se pueden usar

para crear bifurcaciones dentro del código de sus programas, de modo que se ejecuten unas instrucciones u otras diferentes de acuerdo con ciertas condiciones.

Esto permite que un programa actúe de forma diferente cada vez que se ejecuta, normalmente dependiendo de los datos que lea del sistema de entrada/salida.

A continuación, en el apartado “Estructuras de repetición” se explica cómo puede realizar bucles de instrucciones dentro del código fuente, de forma que un mismo bloque se ejecute un número consecutivo en ocasiones o hasta que se cumpla alguna condición concreta.

Esto simplifica enormemente algunas partes del código o, al menos, lo hace mucho más corto, y permite poder volver a realizar ciertas acciones sin tener que volver a ejecutar el programa de nuevo desde el principio.

A lo largo de la unidad, como hilo argumental, se partirá siempre del análisis de un ejemplo en el que se ve claramente cómo varía el comportamiento de un programa en diferentes ejecuciones. Nuevamente, después de exponer temas importantes, le proponemos retos a realizar para entenderlos mejor.

Ahora bien, debe tener en cuenta que, aunque los ejemplos estarán centrados en la sintaxis específica del lenguaje Java, los conceptos explicados son genéricos de la inmensa mayoría de lenguajes de programación. Sólo tendrá que ver cuál es la sintaxis exacta en cada caso, pero la estructura básica suele ser la misma. Por tanto, al igual que en la gestión de variables, debe tener presente que estos conceptos van más allá del lenguaje Java.

## Resultados de aprendizaje

Al finalizar esta unidad, el alumno/a:

1. Utiliza correctamente tipos de datos simples y compuestos utilizando las estructuras de control adecuadas.





## 1. Algoritmos

El desarrollo de una aplicación informática se puede dividir en un conjunto de fases que toman como punto de partida la detección de la necesidad de realizar un programa, y que finalizan cuando se comprueba que éste ya funciona correctamente.

Desde un punto de vista simplista, una vez que el programa está siendo utilizado, se podría considerar que el trabajo del desarrollador ha terminado. Ahora bien, muchas veces puede darse la circunstancia de que durante el uso se detecten posibles cambios o mejoras que debería hacerse. En este caso, será necesario recular a alguna de las fases previas y trabajar hasta disponer de una nueva versión del programa.

En este último caso, aparece un ciclo que hace que la aplicación informática se encuentre en continua evolución. Por eso suele hablarse del ciclo de vida de una aplicación informática.

En este apartado se explicará en qué consiste cada una de las fases de este ciclo de vida, enumeradas a continuación:

1. Definición del problema.
2. Diseño del algoritmo.
3. Implementación del programa.
4. Verificación y pruebas.
5. Puesta en marcha y mantenimiento.

### 1.1 Definición del problema

La definición del problema es el primer paso del proceso a seguir para resolverlo. Hay que tener muy claro qué es lo que debe resolverse. Y esto implica una muy buena comunicación con la organización que encarga la realización del programa y con los usuarios finales, estableciendo con precisión y claridad los objetivos a alcanzar.

Normalmente, el punto de partida se establece cuando un cliente (ya sea usted mismo o una tercera persona), detecta que necesita resolver varias veces un problema de cierta complejidad y decide que será mucho más fácil si puede automatizar las operaciones mediante un programa. Llegados a esta conclusión, es necesario empezar por la realización de un análisis del problema, que en ocasiones puede llevar bastante tiempo, ya que el problema puede ser difícil de comprender o el cliente puede no tener del todo claro qué quiere.

Una vez que el problema está definido, es necesario llevar a cabo un análisis funcional. Ésta no es estrictamente responsabilidad del programador, por lo que no se hará énfasis en este apartado, pero esto no implica que no tenga que ser conocedor. A fin de cuentas, los programadores serán los encargados de desarrollar los programas resultantes de esta etapa.

En este análisis, se suele hacer lo siguiente:

- Hablar con todos los usuarios implicados de forma que logremos tener todos los puntos de vista posibles sobre el problema. Si ustedes son los usuarios el trabajo es más sencillo, evidentemente.
- Dar todas las soluciones posibles con un estudio de viabilidad, considerando los costes económicos (material y personal).
- Proponer al cliente, futuro usuario de la aplicación, una solución entre las posibles, y ayudarle a tomar la decisión más adecuada.
- Romper la solución adoptada en partes independientes para dividir las tareas entre un equipo de trabajo.

Como resultado, se dispondrá de una idea clara de qué hacer y con qué recursos. Sólo entonces es el momento de ver cómo hacerlo, en forma de una aplicación informática.

## 1.2 Diseño del algoritmo

Una vez establecido cuál es el problema a resolver, es el momento de detenerse a reflexionar sobre qué estrategia habrá que seguir para resolverlo mediante un programa y qué forma deberá tomar el código fuente. Para establecerlo, es necesario recordar claramente qué puede hacer un ordenador (qué tipo de órdenes puede aceptar) y qué hay dentro de un programa. En cuanto a lo que puede hacer un ordenador, básicamente se trata de procesar datos. En cuanto a la composición de un programa, se puede considerar que existen dos elementos fundamentales que colaboran para llevar a cabo la tarea encomendada. Por un lado, están los datos a procesar, ya sea directamente en forma de literales o introducidos por el usuario. Por otro lado, el conjunto de órdenes que se da al ordenador, las instrucciones, a fin de que se produzca este proceso de transformación.

Qué es un programa

Una definición clásica es:

Programa = datos + algoritmo.

Dentro de un programa se representará lo que se conoce formalmente como algoritmo: un mecanismo para resolver un problema o tarea concreta, descrito como una secuencia finita de pasos.

Aunque todos los programas contienen algoritmos, traducidos en una serie de instrucciones, este concepto no está ligado exclusivamente a la programación. Cualquier descripción compuesta por una serie de pasos que deben seguirse ordenadamente para lograr una

hito es un algoritmo. Fuera del mundo de los ordenadores uno de los ejemplos más claros son las recetas de cocina. Por ejemplo, el siguiente conjunto de pasos podría ser el algoritmo para hacer un par de huevos fritos:

1. Coger dos huevos de la nevera.
  - (a) Si no hay, ir de compras.
2. Coger sal y aceite del armario.
  - (a) Si no hay, ir de compras.
3. Poner aceite en la sartén.
4. Poner la sartén al fuego.
5. Mientras el aceite no esté caliente, es necesario esperar.
6. Por cada huevo se debe hacer lo siguiente:
  - (a) Romperlo y poner el contenido en la sartén.
  - (b) Poner sal.
7. Mientras los huevos no estén fritos, es necesario esperar.
8. Quitar los huevos de la sartén y servirlos.

En cualquier caso, lo primordial es que, tanto para establecer qué debe hacer una persona como un ordenador, antes hay que pensar exactamente cómo dividir la tarea en acciones individuales y qué orden deben seguir. Por tanto, una de las tareas primordiales del programador antes de ni siquiera empezar a escribir código fuente es dedicar un tiempo a reflexionar ya diseñar un algoritmo que sirva para llevar a cabo la tarea dada por la definición del problema. Esta fase es muy importante, ya que va a condicionar totalmente la fase de implementación. Si el algoritmo es incorrecto, habrá perdido tiempo y esfuerzo creando un código fuente que no hace lo que desea.

### 1.2.1 Programación estructurada

Existen diferentes aproximaciones para diseñar algoritmos. La que aprenderá en este módulo es la llamada programación estructurada. En ésta, los pasos de un algoritmo se dividen en diferentes bloques de instrucciones, cada uno elegido únicamente entre los tres tipos de estructura siguientes:

- Estructura lineal o secuencial: las instrucciones se ejecutan en el mismo orden en el que se han escrito. Éste es el que ha visto hasta ahora. En el ejemplo del huevo frito, equivaldría a la transición entre el paso 5 y 6. Uno siempre va detrás del otro, secuencialmente.

- Estructura de selección o condicional: existen ciertas condiciones que provocan la ejecución de bloques de instrucciones diferentes dependiendo de si se cumplen o no estas condiciones. En el ejemplo del huevo frito, éste es el caso de los pasos 1 o 2, en los que, dada una condición, la acción es diferente. Dependiendo de si hay huevos en la nevera, se pueden coger o se necesitará ir a comprarlos.
- Estructura de repetición o iterativo: el bloque de instrucciones se ejecuta un número finito en ocasiones, ya sea un número concreto o hasta que se cumple una condición. En el caso del huevo frito, el primer caso de esta estructura se ve en el paso 6, en el que hay que repetir las mismas órdenes un cierto número de veces (dos, una por cada huevo). El segundo supuesto ocurre en los pasos 5 y 7, en los que hay que realizar la acción de esperar mientras no se dé una condición concreta.

En un ordenador: abrir un archivo de texto

Cuando ejecute un editor de texto y seleccione abrir un archivo, seguro que el ordenador debe realizar un conjunto de tareas una detrás de otra antes de poder editar el texto. Primero le preguntará dónde se encuentra el archivo en cuestión. Luego lo busca y lo carga. Y finalmente procesa los datos para que sean mostrados correctamente en la pantalla. Cada paso puede ser más o menos complejo, pero está claro que deben seguir un orden y no se puede avanzar al siguiente hasta finalizar el anterior. Se hacen de forma secuencial.

En un ordenador: las opciones de un editor de texto

Cuando ejecute un editor de texto, seguro que verá que hay un gran número de acciones diferentes que puede llevar a cabo: abrir un documento, crear uno nuevo, guardar el documento actual, etc. Cada una de estas opciones se ve claramente diferenciada del resto, ya que suelen estar asociadas a elementos gráficos independientes, dentro de un menú o una barra de herramientas. Lo que está claro es que el conjunto de instrucciones a ejecutar en cada caso será diferente, ya que cada opción hace una tarea diferente. Por tanto, es necesario un mecanismo para establecer qué bloque de instrucciones concreto se ejecuta cada vez que seleccione cada opción.

En un ordenador: las opciones de un editor de texto

A la hora de imprimir un documento en un editor de texto, a menudo puede solicitar cuántas copias se desea enviar a la impresora. Este número de copias puede ser desde una hasta un valor cualquiera, hasta el límite de las hojas de papel y tinta de que dispone. De entrada, se puede establecer que la acción de imprimir un único documento se basará en un bloque de instrucciones concreto. Cada vez que se ejecutan estas instrucciones, el equipo imprime una copia del documento.

Por tanto, una manera sencilla de llevar a cabo muchas copias de un documento sería simplemente repetir tantas veces como copias queramos el conjunto de instrucciones que se usaría para imprimir sólo una.

Estos tres tipos de estructura, denominadas a escala general estructuras de control, permiten establecer qué instrucciones de su programa debe ejecutarse en cada momento y especificar el orden en que se ejecutan las instrucciones y qué caminos siguen en el proceso de ejecución. Es decir, establecer cuál es el flujo de control del programa.

Por el momento, sólo ha tenido contacto con la estructura secuencial, a la que corresponden todos los ejemplos vistos hasta ahora. En los siguientes apartados verá cómo utilizar las estructuras de selección y repetición.

## 1.2.2 Representación de algoritmos

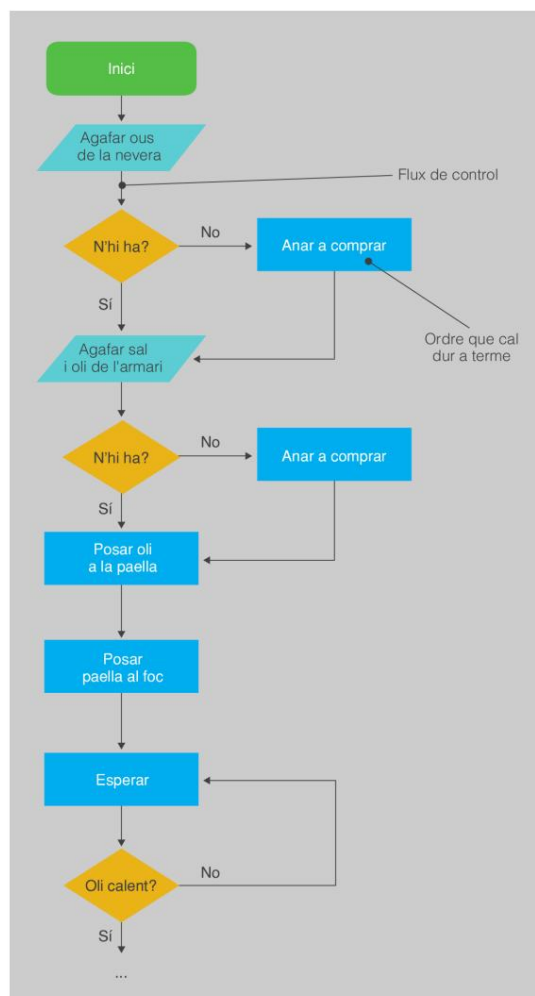
Un algoritmo puede ser representado mediante muchos tipos de notaciones. Una forma muy simple es con una lista de frases expresadas con sus palabras (lenguaje natural), tal y como se ha visto en el ejemplo del huevo frito. Ésta es una aproximación suficiente para hacerse una idea de las tareas a realizar. Ahora bien, también es un sistema que puede ser ambiguo y resultar confuso para algoritmos complejos. Existen otros mecanismos de notación más formal, especialmente usados cuando el algoritmo para describir formará parte de un programa de ordenador. Entre ellos se encuentran el pseudocódigo, los diagramas de flujo de control o, directamente, los códigos fuente de los diferentes lenguajes de programación.

### Pseudocodi

El pseudocódigo es un lenguaje informal de alto nivel que utiliza las convenciones y la estructura de un lenguaje de programación, pero que está orientado a ser entendido por los humanos.

A partir de ahora, para esquematizar el funcionamiento de ciertas estructuras de control se usarán diagramas de control de flujo, mientras que para representar algoritmos completos, se utilizará directamente código fuente en lenguaje Java.

Figura 1.1. Aspecto de un diagrama de flujo de control



Un diagrama de flujo de control consiste en una subdivisión de pasos secuenciales, de acuerdo con las sentencias y estructuras de control de un programa, que muestra los distintos caminos que puede seguir un programa a la hora de ejecutar sus instrucciones. Cada paso se asocia a una figura geométrica específica.

Lo usaremos exclusivamente para aclarar su funcionamiento. No se explicará con detalle todo el conjunto de símbolos que se pueden utilizar. Tampoco se seguirá el formato hasta el último detalle. Es suficiente con disponer de unos esquemas sencillos que sirvan para darle una idea clara del flujo de control del programa para cada estructura usada, representada de forma visual. A título meramente ilustrativo, para que tenga un primer contacto del aspecto que tienen, la figura 1.1 muestra el diagrama de flujo de control de parte del algoritmo para hacer huevos fritos. Observe cómo las figuras geométricas especifican las instrucciones que se van ejecutando y cómo las flechas establecen el orden de ejecución de las instrucciones: el flujo de control. En el caso de la descripción del algoritmo de un programa, el contenido de cada figura, el orden a seguir, corresponderá a las instrucciones del programa.

### 1.3 Implementación del programa

Una vez definido el algoritmo, es necesario convertirlo a código fuente en un archivo, de acuerdo con un lenguaje de programación concreto, para obtener un archivo ejecutable. Según la notación utilizada para representar el algoritmo diseñado, esto será más fácil o más complicado. Evidentemente, si el algoritmo se ha representado mediante la sintaxis de un lenguaje concreto, convertirlo a código fuente es muy fácil, puesto que es un sencillo ejercicio de enganchar y copiar.

Durante esta etapa, es importante que todo el proceso esté bien documentado, para que cualquier otro desarrollador, incluso el propio programador cuando ya ha pasado un cierto tiempo, pueda retocar la aplicación si es necesario. Piense que en un programa informático se automatiza una forma de actuación, lo que implica reflejar un cierto conocimiento. Pero el conocimiento no es único y, por tanto, entender lo que otro desarrollador o uno mismo decidió en el pasado es muy difícil. La documentación es la única forma de garantizar este conocimiento. Por tanto, lo mínimo que se puede pedir cuando se implementa un programa es que se comente el código fuente de forma que sea más comprensible. En este aspecto, se espera que al menos se explique con comentarios que hace el programa y los blogs de instrucciones que se consideren de difícil comprensión.

### 1.4 Verificación y pruebas

Antes de que un programa se pueda considerar finalizado y se ponga a disposición del cliente, será necesario detectar posibles errores y comprobar que actúa como se espera. Hi

hay muchas estrategias para probar la corrección de un programa, algunas muy complejas. Para este módulo, es suficiente con una aproximación relativamente simple pero más que efectiva: usar juegos de pruebas.

Un juego de pruebas es un conjunto de situaciones o entradas de datos que permiten probar la actuación del programa. Este conjunto debería ser completo, en el sentido de que debe abarcar todas las posibilidades reales.

Primero será necesario diseñar y preparar el juego de pruebas para ejecutar el programa posteriormente en las diferentes situaciones preparadas. De esta forma se puede comprobar su buen funcionamiento en todo momento. En caso de que siempre funcionen, puede estar más o menos seguros de la fiabilidad del programa. Ahora bien, la realidad es que siempre se producen situaciones no previstas que se acabarán detectando cuando el programa esté ya en marcha y siendo utilizado por el cliente. Seguro que alguna vez se han encontrado con programas comerciales, como algunos sistemas operativos, que a pesar de estar a la venta y considerados finalizados, se comportaban anómalamente en condiciones normales (se cuelgan, reinician el ordenador, etc.). En programas complejos, resulta difícil detectar hasta el último error posible.

Normalmente, cuando un programa no actúa correctamente, el problema puede estar en lo siguiente:

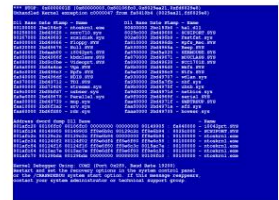
- El diseño del algoritmo.
- La codificación, que es semánticamente incorrecta (pero con sintaxis correcta).

A veces el error se encuentra de forma sencilla con un simple seguimiento del algoritmo diseñado, observando si las variables se modifican correctamente y toman los valores esperados dados unos datos iniciales concretos. Pero la mayoría de las veces el error cuesta encontrar porque se parte de la premisa de que el algoritmo es correcto, lo que no necesariamente debe ser cierto. En este caso, un mecanismo invaluable para encontrar el error es llevar a cabo la depuración del programa.

Un programa depurador (debugger, en inglés) es un programa que permite:

- Ejecutar un programa instrucción por instrucción y ver qué ocurre después de la ejecución de cada una
- Obtener el valor de los datos antes y después de ejecutar una instrucción
- Modificar el valor de los datos durante la ejecución
- Interrumpir o detener la ejecución del programa en cualquier punto

La mayoría de entornos de programación actuales disponen de herramientas de depuración. Si no es el caso, el programador no tendrá más remedio que realizar la depuración manualmente. Este método consiste en intercalar temporalmente en medio del código del programa instrucciones que muestren por pantalla los valores de las variables,



Un comportamiento erróneo clásico: la pantalla azul del sistema operativo Windows. Imagen de Wikimedia Commons.

Dispone de un anexo donde le explicamos el funcionamiento del depurador incluido en el IDE que usará a lo largo del curso.

El término original debug significa literalmente 'eliminar luciérnagas'.

Coloquialmente, estas instrucciones para mostrar valores de variables se llaman "chivatos".

con el fin de realizar el seguimiento de la ejecución y localizar la causa del problema. Para ello, se pueden utilizar las instrucciones que aporte el lenguaje para mostrar datos en la pantalla, como `System.out.println(...)` en Java. Una vez que el programa funciona correctamente, habrá que quitar el código, o al menos convertir estas instrucciones en comentarios.

### 1.5 Puesta en marcha y mantenimiento

Una vez se tiene el programa final, supuestamente libre de errores, es necesario instalarlo siguiendo las instrucciones del entorno de trabajo y ponerlo en funcionamiento. Esto es lo que formalmente se llama la fase de explotación.

Parece que llega al final del proceso, pero todavía nos queda una acción: formar al usuario. Es necesario tener paciencia. Piense que usted ha estado mucho tiempo pensando en el problema y trabajando en la solución. Para usted es la solución normal, la tiene asumida, pero para el usuario no es necesariamente la solución deseada o esperada. Por tanto, habrá que tener una buena dosis de paciencia para explicar el funcionamiento de la aplicación. Esta formación debe estar siempre acompañada de la guía o manual del usuario. No debe ser un libro de lectura difícil; deberían ser los apuntes que le permitieran moverse rápidamente por la aplicación, tratando todas las posibilidades de una forma rápida y clara.

Llegados a este punto ya se dispone a descansar, pero puede tener la mala suerte de que con el uso de la aplicación se llegue a la conclusión de que el programa no hace exactamente lo que se espera o se detectan comportamientos erróneos que han eludido la fase de pruebas. Desde una perspectiva más positiva, también puede que en esta fase el cliente esté muy satisfecho y detecte que estaría bien que, ahora que ya tiene una aplicación funcional, le vuelva a contratar para añadir nuevas funcionalidades. En este caso, será necesario retroceder hasta la fase correspondiente y realizar las modificaciones pertinentes. Empieza una nueva iteración en el ciclo de vida de la aplicación.



## 2. Estructuras de selección

Entre los distintos tipos de estructuras de control que permiten establecer el flujo de control de un programa, las más fáciles de entender son aquellas que crean bifurcaciones o caminos alternativos, de modo que, según las circunstancias, se ejecute un conjunto de instrucciones u otro. De esta forma, dadas diferentes ejecuciones de un mismo código fuente, parte de las instrucciones que se ejecutan pueden ser diferentes para cada caso.

Las estructuras de selección permiten tomar decisiones sobre qué conjunto de instrucciones se deben ejecutar en un punto del programa. O sea, seleccionar qué código se ejecuta en un momento determinado entre caminos alternativos.

Toda estructura de selección se basa en la evaluación de una expresión que debe dar un resultado booleano: true (cierto) o false (falso). Esta expresión se llama la condición lógica de la estructura.

El conjunto de instrucciones que se ejecutará dependerá del resultado de la condición lógica, actuando como una especie de interruptor que marca el flujo a seguir dentro del programa. Normalmente, esta condición lógica se basa en parte, o en su totalidad, en valores almacenados en variables cuyo valor puede ser diferente para diferentes ejecuciones del programa. De lo contrario, no tiene sentido utilizar una estructura de selección, ya que mientras se está escribiendo el programa ya se puede predecir cuál será el resultado de la expresión. Por tanto, como siempre será lo mismo, siempre se ejecutarán las mismas instrucciones sin que haya bifurcación posible.

Existen distintos modelos de flujos alternativos a la hora de ejecutar instrucciones, aunque para todos se usa la misma familia de sentencias y una estructura similar en el código fuente. Por tanto, todos los aspectos destacados para uno de los modelos se aplican también a todos los demás. También hay que decir que si bien los ejemplos y la sintaxis descrita en este módulo se centran en el lenguaje Java, la mayoría de las estructuras de selección descritas son compartidas con los otros lenguajes de programación, por lo que el concepto general es aplicable más allá del Java. Sólo tendrá que buscar en la documentación la sintaxis específica para el lenguaje elegido.

### 2.1 Una desviación temporal del camino: selección simple

El caso más simple dentro de las estructuras de selección es aquel en el que existe un conjunto o bloque de instrucciones que sólo desea que se ejecuten bajo unas circunstancias concretas. De lo contrario, este bloque es ignorado y, desde el punto



La selección simple, una desviación temporal en el camino de instrucciones.  
Imagen de Wikimedia Commons

de vista de la ejecución del programa, es como si no existiera. Un ejemplo sería el programa de una tienda virtual que aplica un descuento al precio final de acuerdo con cierto criterio (por ejemplo, si la compra total es como mínimo de 100 AC). En este caso, existe un conjunto de instrucciones, las que aplican el descuento, que sólo se ejecutan cuando se cumple la condición. De lo contrario, se ignoran y el precio final es el mismo que el original.

La estructura de selección simple permite controlar que se ejecute un conjunto de instrucciones si y sólo si se cumple la condición lógica (es decir, el resultado de evaluar la condición lógica es igual a true). De lo contrario, no se ejecutan.

### 2.1.1 Sintaxis y comportamiento

Para llevar a cabo este tipo de control sobre las instrucciones del programa, es necesario usar una sentencia if (si...). En el caso de Java, la sintaxis es la siguiente:

```
1 instrucciones del programa 2 if (expresión
booleana) {
3     Instrucciones para ejecutar si la expresión evalúa a true (cierto) - Bloque if
4 } 5
resto de instrucciones del programa
```

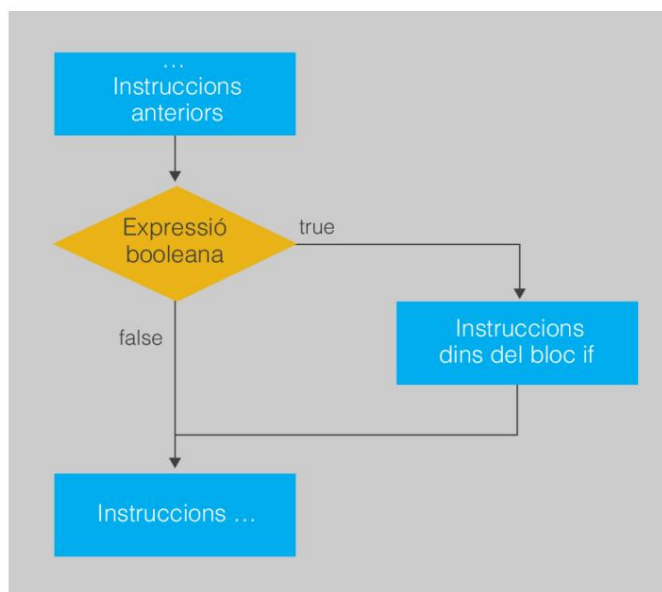
si

El nombre de la sentencia if básicamente dice: "Si se cumple cierta condición, haz esto...".

Si entre los paréntesis se pone una expresión que no evalúa un resultado de tipo booleano, habrá un error de compilación.

La figura 2.1 muestra un diagrama del flujo de control de esta sentencia, que establece los distintos bloques de instrucciones que se ejecutan en cada caso, dependiendo del resultado de evaluar la expresión booleana.

Figura 2. 1. Diagrama de flujo de control para una selección simple



Un diagrama de flujo de control consiste en una subdivisión de pasos secuenciales, de acuerdo con las sentencias y estructuras de control de un programa, que muestra los distintos caminos que puede seguir el programa a la hora de ejecutar las instrucciones. Cada paso se asocia con una figura geométrica específica.

A partir de ahora, para esquematizar el funcionamiento de las estructuras de control se utilizará este tipo de diagramas. Sólo lo usaremos para aclarar su funcionamiento, por lo que no se explicará con detalle todo el conjunto de símbolos que se pueden usar. A la hora de usarlos tampoco se seguirá el formato hasta el último detalle. Es suficiente con disponer de unos esquemas sencillos que sirvan para darle una idea clara del flujo de control del programa para cada estructura usada de forma visual.

---

Una posible bifurcación en el camino de un flujo de control se indica con un rombo que contiene una expresión a evaluar.

---

### 2.1.2 Ejemplo: calcular un descuento

Se quiere realizar un programa que aplique un descuento a un precio dependiendo de su valor. Para ver claramente que hay diferentes caminos dentro de las instrucciones, primero estableceremos cuáles son las tareas que debe realizar el programa y en qué orden.

El programa debería hacer:

1. Decidir cuál es el valor mínimo para optar al descuento y cuánto se descontará.
2. Pedir que se introduzca el precio inicial, en euros, por el teclado.
3. Llegir-lo.
4. Ver si el precio introducido es igual o mayor que el valor mínimo para optar al descuento.
  - (a) Si es así, se aplicará el descuento sobre el precio inicial.
5. Mostrar el precio final.

---

Recuerde que puede que Java lea los enteros introducidos por teclado con los decimales indicados con coma, y no punto, según la configuración local.

---

En el paso 4 se puede observar que es necesario tomar una decisión de acuerdo con una condición y que alguna de las tareas sólo se hace si ésta se cumple (el paso I). Por tanto, queda establecido que es necesaria una estructura de selección simple. Partiendo de ahí, un posible código fuente que correspondería a este esquema sería el siguiente. Observe qué instrucciones se corresponden a cada uno de los pasos descritos.

A continuación se muestra el código fuente que realiza estas tareas. Compílelo, ejecútelo y observe cómo el resultado final mostrado por pantalla es diferente según el valor que introduzca por el teclado.

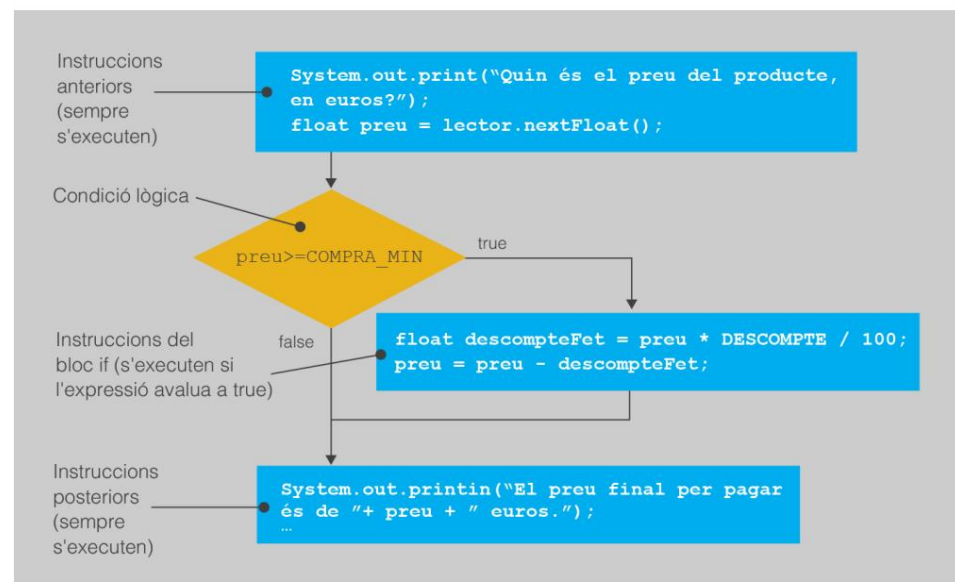
```

1 import java.util.Scanner;
2 //Un programa que calcula descuentos.
3 public class Descuento {
4     //NO 1
5     //Se realiza un descuento del 8%.
6     public static final float DESCUENTO = 8;
7     //Se hace descuento por compras de un mínimo de 100 euros.
8     public static final float COMPRA_MIN = 100;
9     public static void main (String[] args) {
10         Scanner lector = new Scanner(System.in);
11         //NO 2 i 3
12         System.out.print("¿Cuál es el precio del producto, en euros? ");
13         float precio = lector.nextFloat();
14         lector.nextLine();
15         //NO 4
16         //Estructura de selección simple.
17         //Si la expresión evalúa true, ejecuta el bloque en el if.
18         //En caso contrario, ignóralo.
19         if (precio >= COMPRA_MIN) {
20             //YO NO
21             float descuentoFet = precio * DESCUENTO / 100;
22             precio = precio - descuentoFet;
23         }
24         //NO 5
25         System.out.println("El precio final por pagar es de " + precio + " euros.");
26     }
27 }

```

En la figura 2.2 se representa el diagrama de flujo de control de este programa en concreto. En la figura se visualiza a qué parte del diagrama corresponde cada parte del código.

Figura 2. 2. Diagrama de flujo de control para el programa de cálculo de un descuento



Reto 1: modifique el programa para que, en lugar de hacer un descuento del 8% si la compra es de 100 AC o más, aplique una penalización de 2 AC si el precio es inferior y 30 AC.

### 2.1.3 Aspectos importantes de la selección simple

La introducción de estructuras de selección en un programa complica la sintaxis del código fuente, por lo que es necesario ser muy cuidadoso a la hora de escribir una sentencia if. Si no lo hace, o bien el compilador dará error, o bien el programa no se comportará correctamente, y deberá repasar el código fuente para ver dónde está el error. Cabe decir que el segundo caso suele ser mucho más molesto que el primero. Por tanto, vale la pena hacer un repaso de los aspectos más relevantes de la selección simple. Estas cuestiones afectan en una u otra medida a todas las estructuras de selección.

- La expresión booleana que denota la condición lógica puede ser tan compleja como se quiera, pero debe estar siempre entre paréntesis.
- Las instrucciones a ejecutar si la condición es cierta están englobadas entre dos claves ({, }). Este conjunto se considera un bloque de instrucciones asociado a la sentencia if (bloque if ).
- La línea donde están las claves o la condición no termina nunca en punto y coma (;), al contrario que otras instrucciones.
- Aunque no es imprescindible, es una buena costumbre que las instrucciones del bloque estén sangradas.

Visto esto, merece la pena remarcar que con la introducción de estructuras de selección simple también aparece por primera vez código fuente con diferentes bloques de instrucciones: el del método principal y los asociados a la sentencia if. La principal característica de este hecho es que la relación entre bloques es jerárquica: todos los nuevos bloques de instrucciones son subbloques del método principal. La figura 2.3 muestra los distintos bloques existentes en el ejemplo.

Figura 2.3. Bloques de instrucciones en el programa de ejemplo.

```
//Un programa que calcula descomptes
public class Descompte {

    //Es fa un descompte del 8%
    public static final float DESCOMPTE = 8;

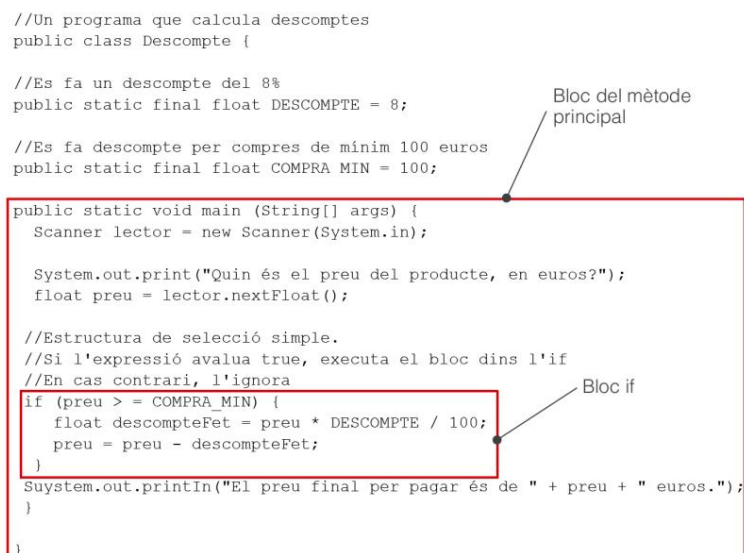
    //Es fa descompte per compres de minim 100 euros
    public static final float COMPRA_MIN = 100;

    public static void main (String[] args) {
        Scanner lector = new Scanner(System.in);

        System.out.print("Quin és el preu del producte, en euros?");
        float preu = lector.nextFloat();

        //Estructura de selecció simple.
        //Si l'expressió avalua true, executa el bloc dins l'if
        //En cas contrari, l'ignora
        if (preu >= COMPRA_MIN) {
            float descompteFet = preu * DESCOMPTE / 100;
            preu = preu - descompteFet;
        }

        System.out.println("El preu final per pagar és de " + preu + " euros.");
    }
}
```



Como irá viendo, esta circunstancia se repetirá en todas las estructuras de control de un programa. Por eso debe tener muy claro dónde empieza y acaba cada blog, y qué blogs son subbloques de otro.

También vale la pena hacer énfasis en que, cuando se usen estructuras que delimitan bloques de instrucciones diferentes, identificados para estar escritos entre claves ({, }), es importante sangrar cada línea. De esta forma, se facilita la legibilidad del código fuente y la identificación de cada bloque de instrucciones. Si se fija, verá que, de hecho, Esto ya se había aplicado hasta ahora a las declaraciones dentro del método principal (también delimitadas por claves), respecto a la declaración de inicio de la clase. Todas las instrucciones del método principal están sangradas respecto al margen de la declaración del método principal.

A modo de resumen, la tabla 2.1 muestra una pequeña lista de errores típicos que se pueden cometer al usar una sentencia if.

Tabla 2. 1. Errores típicos al usar una sentencia "if"

| Mensaje de error al compilar                                                             | Error cometido                                     | Ejemplo                        |
|------------------------------------------------------------------------------------------|----------------------------------------------------|--------------------------------|
| '(' esperado                                                                             | Expresión no rodeada de paréntesis                 | yf precio >= COMPRA_MIN) {     |
| ',' esperado                                                                             | Falta; en alguna instrucción del bloque si         | precio = precio - descuentoFet |
| Siempre ejecuta blog if                                                                  | Se ha puesto ; en la sentencia if                  | if (precio >= COMPRA_MIN); {   |
| Sólo hace condicionalmente la primera instrucción del blog if, el resto las hace siempre | Faltan las claves que han de rodear el blog, {...} | if (precio >= COMPRA_MIN)      |

Vinculado al último error, cabe mencionar que cuando el blog if sólo tiene una única instrucción, el uso de claves es opcional. De todas formas, es muy recomendable usarlas siempre, independientemente de este hecho. Esto facilita la identificación del blog de código asociado a la sentencia if cuando se está leyendo el código fuente.

```

1 yf (precio >= COMPRA_MIN)
2     precio = precio - (precio * DESCUENTO / 100);
3
4 es equivalente a
5
6 if (precio >= COMPRA_MIN) {
7     precio = precio - (precio * DESCUENTO / 100);
8 }
```

Ahora bien, ¡alerta!

```

1 yf (precio >= COMPRA_MIN)
2     float descuentoFet = precio * DESCUENTO / 100;
3     precio = precio - descuentoFet;
4
5 es equivalente a (fijaos donde están ubicadas las claves)
6
7 if (precio >= COMPRA_MIN) {
8     float descuentoFet = precio * DESCUENTO / 100;
9 }
10 precio = precio - descuentoFet;
```

## 2.2 Dos caminos alternativos: la sentencia "if/else"

Suponga que ahora desea realizar un programa en el que se debe intentar adivinar un número entre 1 y 10. En este caso, ya diferencia del anterior, ahora hay dos escenarios excluyentes: o se ha adivinado el número, o no se ha adivinado. Según el caso que se trate, la respuesta del programa debe ser diferente. Por tanto, además de código común para cualquier caso y de un bloque que especifique las acciones para llevar a cabo si se cumple la condición, ahora es necesario otro que indique qué hacer sólo en caso contrario. Para poder hacer esto tenemos la estructura de selección doble, la sentencia `if/else` (si... si no...).



La doble selección, una bifurcación en el camino. Imagen de Wikimedia Commons

La estructura de selección doble permite controlar el hecho de que se ejecute un conjunto de instrucciones, sólo si se cumple la condición lógica, y que se ejecute otro, sólo si no se cumple la condición lógica/ Imagen de Wikimedia Commons

Es importante recordar que ambos bloques de código son excluyentes. Nunca puede ocurrir que ambos se acaben ejecutando.

### 2.2.1 Sintaxis y comportamiento

Para llevar a cabo este tipo de control sobre las instrucciones del programa, es necesario usar una sentencia `if/else` (si... si no...). En el lenguaje Java, la sintaxis es la siguiente:

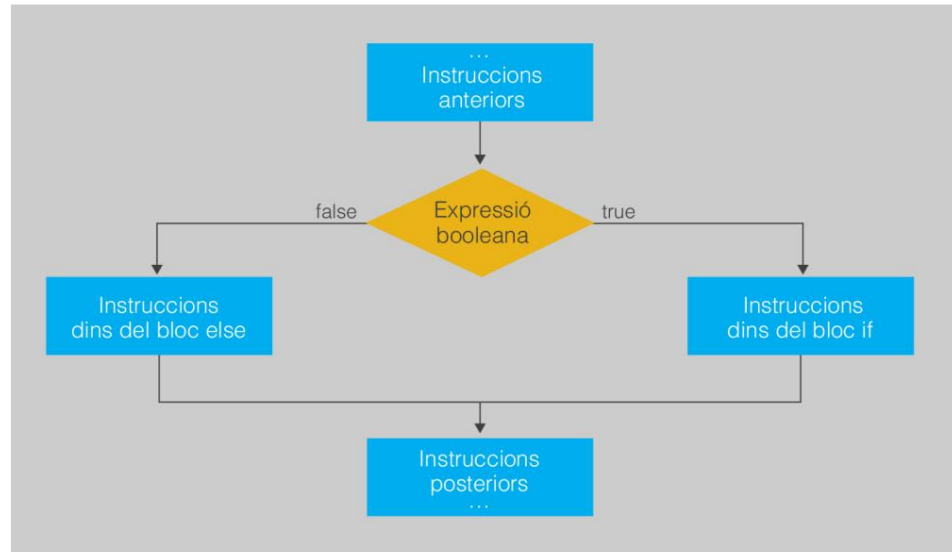
```
1 if (expresión booleana) {  
2     Instrucciones para ejecutar si la expresión la evalúa a true (cierto) – Bloc if 3 } else {  
  
4     Instrucciones alternativas para ejecutar si la expresión la evalúa a false (falso)  
        – Bloquear en caso contrario  
5 }
```

El bloque `if` tiene exactamente las mismas características que cuando se usa en una estructura de selección simple. Éstas se cumplen también para el caso del bloque `else`, con la particularidad de que no tiene ninguna expresión asignada. Simplemente, cuando la condición lógica de la sentencia `if` no se cumple se ejecutan las instrucciones del bloque `else`. La figura 2.4 muestra su diagrama del flujo de control a escala genérica.

si/entonces

La sentencia `if/else`  
básicamente dice: "Si se cumple cierta condición, haz esto... y si no, eso otro...".

Figura 2. 4. Diagrama de flujo de control de una selección doble



Si por algún motivo no se pone instrucción alguna dentro del bloque else y se deja un espacio vacío entre las dos claves, la selección doble se comporta igual que la selección simple. En este caso, es mejor usar sólo la sentencia if en lugar de if/los.

### 2.2.2 Ejemplo: adivina el número secreto

Un posible código de un programa en el que se intenta adivinar un número sirve para mostrar con mayor claridad cómo se puede usar una sentencia if/else. Nuevamente, antes de verlo vale la pena reflexionar sobre cuáles son las tareas que debería realizar el programa y en qué orden.

Básicamente, el programa debe hacer:

1. Decidir cuál será el número para adivinar.
2. Pedir que se introduzca un número por el teclado.
3. Llegir-lo.
4. Ver si el número introducido es igual al valor secreto:
  - (a) Si es igual que el número pensado, informa que se ha acertado.
  - (b) Si no, informa que se ha fallado.

Queda claro que en el paso 4 es necesario tomar una decisión de acuerdo con una condición. El programa deberá emprender dos vías de acción distintas según si la condición se cumple o no. Por tanto, es necesaria una estructura de selección doble. Partiendo de ahí, un posible código fuente que correspondería a este esquema sería el siguiente. De nuevo, observe qué instrucciones se corresponden a cada uno de los pasos descritos. Compílelo y ejecútelo para ver cómo funciona.



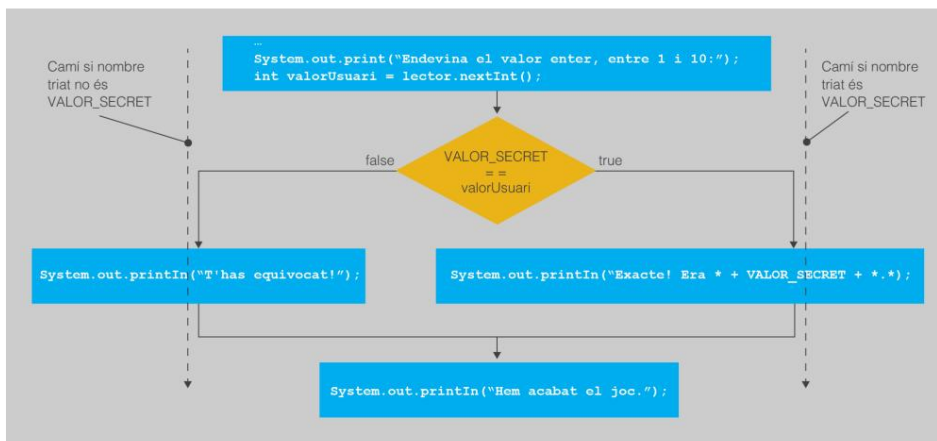
```

1 import java.util.Scanner;
2 //Un programa en el que es necesario adivinar un número.
3 public class Adivina {
4     //NO 1
5     //El número a adivinar será el 4.
6     public static final int VALOR_SECRET = 4;
7     public static void main (String[] args) {
8         Scanner lector = new Scanner(System.in);
9         //NO 2 i 3
10        System.out.println("Empezamos el juego.");
11        System.out.print("Endevina el valor enter, entre 0 i 10: ");
12        int uservalue = reader.nextInt();
13        lector.nextLine();
14        //NO 4
15        //Estructura de selección doble.
16        //O adivina o falla.
17        Si (SECRET_VALUE == uservalue) {
18            //YO NO
19            //Si la expresión evalúa true, ejecuta el bloque en el if.
20            System.out.println("¡Exacto! Era" + VALOR_SECRET + ".");
21        } demás {
22            //NO II
23            //Si la expresión evalúa false, ejecuta el bloque dentro del les.
24            System.out.println("¡Es un error!");
25        }
26        System.out.println("Hemos terminado el juego.");
27    }
28 }

```

A continuación, la figura 2.5 muestra un esquema de cuál es el flujo exacto de ejecución cuándo se pone en marcha el programa y qué papel tiene cada parte del código de acuerdo con la definición de la sintaxis.

Figura 2. 5. Esquema del flujo de control de un programa para adivinar un número



Reto 2: modifique el programa para que, en lugar de un único valor secreto, lo haya dos. Para ganar, basta con acertar uno de los dos. La condición lógica que os será necesario ya no se puede resolver con una expresión compuesta por una única comparación. Será más compleja.

## 2.3 Varios caminos: la sentencia "if/else if/else"



La selección múltiple: múltiples caminos alternativos. Imagen de Wikimedia Commons

Finalmente, a la hora de establecer el flujo de control de un programa, también existe la posibilidad de que haya un número arbitrario de caminos alternativos, no sólo dos. Por ejemplo, imagináis un programa que, a partir de la nota numérica de un examen, debe establecer cuál es la calificación del alumno. Por eso habrá que ver dentro de qué rango se encuentra el número. En cualquier caso, los posibles resultados son más de dos.

La estructura de selección múltiple permite controlar el hecho de que al cumplirse un caso entre un conjunto finito de casos se ejecute el conjunto de instrucciones correspondiente.

### 2.3.1 Sintaxis y comportamiento

Este tipo de control sobre las instrucciones del programa tiene asociada la sentencia `if/else if/else` (en su caso, en este otro caso, en su defecto).

Básicamente, se trata de la misma estructura que la sentencia `if/else`, pero con un número arbitrario de bloques `if` (después del primer bloque, llamados `else if`). En el caso de Java, la sintaxis es la siguiente:

```

1 instrucciones del programa 2 if (expresión booleana
1) {
3     Instrucciones para ejecutar si la expresión 1 la evalúa a true (cierto) - Bloque if
4 } else if (expresión booleana 2) {
5     Instrucciones para ejecutar si la expresión 2 la evalúa a true (cierto) - Bloc else
        si
6 } else if (expresión booleana 3) {
7     Instrucciones para ejecutar si la expresión 3 la evalúa a true (cierto) - Bloc else
        si
8
9 ...se repite tan pronto como sea necesario...
10
11 } else if (expresión booleana N) {
12     Instrucciones para ejecutar si la expresión N evalúa a true (cierto) - Bloc else
        si
13 } else { Instrucciones
14     alternativas si todas las expresiones 1...N se evalúan como falsas (
        fals) - Bloc else
15 } 16
resto de instrucciones del programa

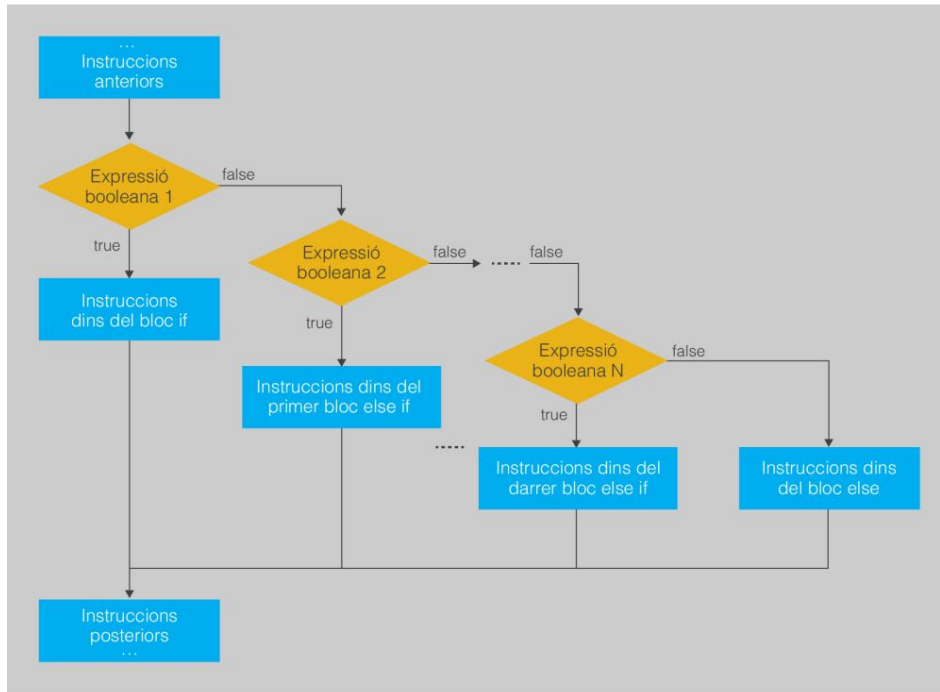
```

si no si no

El nombre de la sentencia `if/else if/else` significa básicamente: "Si se cumple cierta condición, haz esto... Si no, mira si se cumple esto otro... Y si no, eso otro...".

La figura 2.6 muestra un esquema del flujo de ejecución, de acuerdo con el resultado de ir evaluando cada una de las condiciones lógicas. En este caso es algo más complicado, ya que, como puede verse, hay más de una condición lógica dentro de la sentencia. Cada condición se va evaluando ordenadamente, desde la primera hasta la última, y se ejecutará el código del primer caso en el que la expresión evalúe como cierto. Una vez hecho esto, ya no se vuelve a evaluar otra condición restante y se ignora el resto de bloques. Si en este proceso se da el caso de que ninguna de las expresiones evalúa a cierto, se ejecutarán las instrucciones del `else`.

Figura 2. 6. Diagrama de flujo de control de una selección múltiple



Lo importante de esta sentencia es que sólo se ejecutará un único blog de todos los posibles. Incluso en caso de que más de una de las expresiones booleanas pueda evaluar a cierto, sólo se ejecutará el bloque asociado a la primera de estas expresiones dentro del orden establecido en la sentencia.

También es destacable el hecho de que el `blog else` es opcional. Si no queremos, no es necesario ponerlo. En este caso, si no se cumple ninguna de las condiciones, no se ejecuta instrucción alguna entre las incluidas dentro de la sentencia.

### 2.3.2 Ejemplo: transformar evaluación numérica a texto

La forma de usar la sentencia `if/else if/else` puede verse mejor mediante el código del ejemplo propuesto anteriormente. Una vez más, antes de ver el código se repasarán los pasos que debe realizar el programa para llevar a cabo su objetivo y el orden a seguir.

1. Pedir que se introduzca la nota por el teclado.
2. Leerla.
3. Mostrar un texto u otro según el rango de valores dentro del cual se encuentra la usar:
  - (a) Si es mayor o igual que 9 y menor o igual que 10, la nota es de "Excelente".
  - (b) Si es mayor e igual que 6,5 pero estrictamente menor que 9, la nota es "Notable".

(c) Si es mayor e igual que 5 pero estrictamente menor que 6,5, la nota es "Suficiente".

(d) Si no es ninguno de los casos anteriores, la nota es de "Suspendido".

Esta vez es el paso 3 el que presenta diferentes posibilidades según se cumplan ciertas condiciones. Sin embargo, en este caso hay más de un camino (hay cuatro). Además, es necesario decidir si se cumple una condición diferente para decidir cuál es el camino a seguir entre todas las opciones. El último se lleva a cabo simplemente cuando no se cumple ninguno de los anteriores. Por tanto, se trata de una selección múltiple.

---

No olvide que en una estructura de selección la condición lógica siempre va entre paréntesis.

---

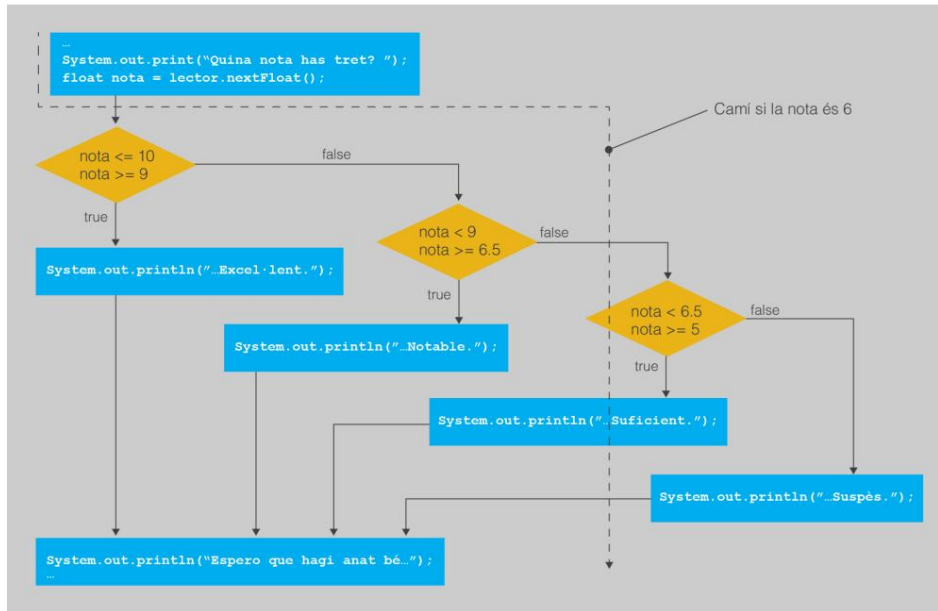
Un hecho destacado en esta descripción de las tareas que debe llevar a cabo programa, al menos respecto a los ejemplos anteriores, es que para cada paso posible deben cumplirse dos condiciones a la vez. El valor de la nota debe estar por debajo de cierto valor y por encima de otro. La expresión de tipo booleano necesario evaluar para ver qué camino hay que seguir debe comprobar ambas cosas: que se cumple una cosa y otra. O sea, es necesaria una expresión lógica basada en la conjunción de dos comparaciones. Una vez más, observe qué instrucciones se corresponden a cada uno de los pasos descritos. Fijese en particular en cómo son condiciones lógicas para cada caso.

Compile y ejecute el siguiente programa para ver cómo la nota mostrada depende del valor introducido:

```
1 import java.util.Scanner;
2 //Un programa que indica la nota en texto a partir del valor numérico.
3 public class Evaluación {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //NO 1 i 2
7         System.out.print("Quina nota has tret? ");
8         float nota = lector.nextFloat();
9         lector.nextLine();
10        //NO 3
11        //Estructura de selecció múltiple.
12        //Se entra en el bloque donde la condició lógica evalúe a true.
13        //Si ninguno lo hace, se entra en el else.
14        System.out.print("Tu nota final es ");
15        si ((nota >= 9)&&(nota <= 10)) {
16            //YO NO
17            System.out.println("Excelente.");
18        } else if ((nota >= 6.5)&&(nota < 9)) {
19            //NO II
20            System.out.println("Notable.");
21        } else if ((nota >= 5)&&(nota < 6.5)) {
22            //PASO III
23            System.out.println("Aprobación.");
24        } demás {
25            //NO ES IV
26            System.out.println("Suspès.");
27        }
28        System.out.println("Espero que haya ido bien...");
29    }
30 }
```

El diagrama de flujo para este programa en concreto se muestra a continuación, en la figura 2.7. En este caso, el código seguirá uno entre cinco caminos posibles, todos disyuntos.

Figura 2. 7. Esquema del flujo de control de un programa de asignar la nota



### Otra posibilidad

El programa que apenas se ha expuesto analiza todas las posibilidades a la hora de evaluar las notas de forma muy estricta. Todas las condiciones posibles están absolutamente disjuntas. Nunca puede darse el caso de que dos evalúen cierto, ya que la nota introducida pertenecerá a uno y sólo a uno de los rangos establecidos. Sin embargo, la selección múltiple acepta que las condiciones no lo sean. En caso de que se cumpla más de una, el bloque de instrucciones que se ejecutará será el de la primera condición que ha evaluado como true, por orden de escritura en el código.

Una vez se tiene algo de experiencia programando, se puede jugar con este hecho, para plantear este programa de otra forma:

1. Pedir que se introduzca la nota por el teclado.
2. Leerla.
3. Mostrar un texto u otro según dentro de qué rango de valores se encuentra la nota:
  - (a) Si es mayor o igual que 9, la nota es "Excelente".
  - (b) Si es mayor o igual que 6,5, la nota es "Notable".
  - (c) Si es mayor o igual que 5, la nota es "Suficiente".
  - (d) Si no es ninguno de los casos anteriores, la nota es "Suspendido".

En este planteamiento, el orden de evaluación de las condiciones es mucho más importante. Para una nota de 9,5 se cumplen todas y cada una de las condiciones. Pero sólo se llevará a cabo la acción relativa a la primera de las enumeradas (en este caso, la l).

El código fuente asociado sería el siguiente, prácticamente idéntico al anterior. Compílelo y ejecútelo para comprobar que el comportamiento es el mismo.

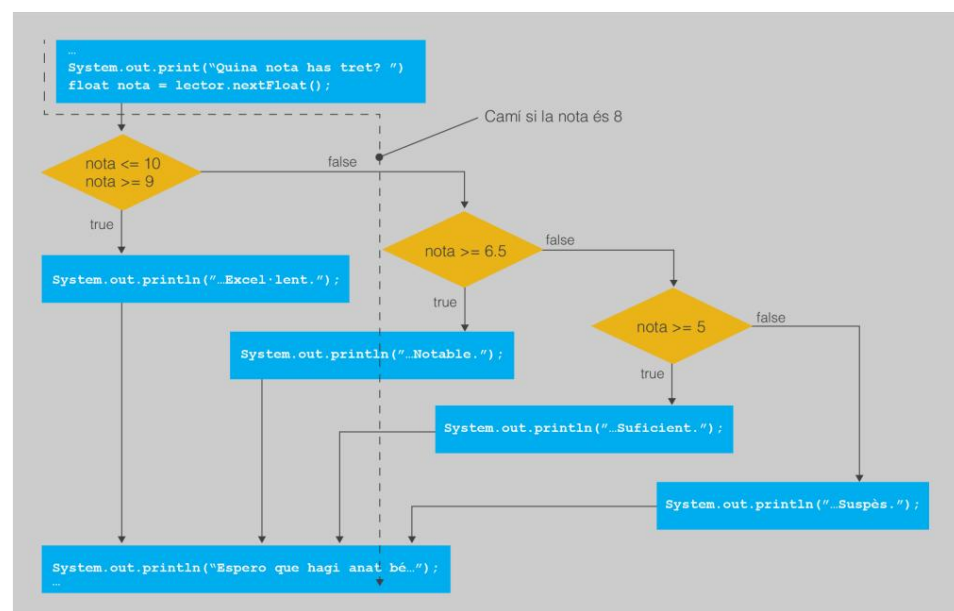
```

1 import java.util.Scanner;
2 //Un programa que indica la nota en texto a partir del valor numérico.
3 public class EvaluaciónSimplificado {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //NO I i 2
7         System.out.print("Quina nota has tret? ");
8         float nota = lector.nextFloat();
9         lector.nextLine();
10        //NO 3
11        //Estructura de selección múltiple.
12        //Se entra en el bloque donde la condición lógica evalúe a true.
13        //Las condiciones se evalúan por orden de aparición.
14        //Si ninguno lo hace, se entra en el bloque else.
15        System.out.print("Tu nota final es ");
16        si ((nota >= 9)&&(nota <= 10)) {
17            //YO NO
18            System.out.println("Excelente.");
19        } else if (nota >= 6.5) {
20            //NO II
21            System.out.println("Notable.");
22        } else if (nota >= 5) {
23            //PASO III
24            System.out.println("Aprobación.");
25        } demás {
26            //NO ES IV
27            System.out.println("Suspès.");
28        }
29        System.out.println("Espero que haya ido bien...");
30    }
31 }

```

Observe atentamente qué ocurre si la nota es 8. En este caso, evalúan como ciertas la segunda ( $\text{nota} \geq 6.5$ ) y la tercera condición ( $\text{nota} \geq 5$ ), pero el bloque de instrucciones que se va a ejecutar es el de la segunda. Esto se ve claramente en el diagrama de la figura 2.8 si va siguiendo ordenadamente el flujo de control establecido. De hecho, si se fija, el diagrama en sí es como el de la figura 2.7, y sólo varían las condiciones lógicas.

Figura 2. 8. Esquema del flujo de control del programa alternativo de asignar la nota



## 2.4 Combinación de estructuras de selección

Las sentencias que definen estructuras de selección son instrucciones como cualesquiera otras dentro de un programa, si bien con una sintaxis algo más complicada.

Por tanto, nada impide que vuelvan a aparecer dentro de bloques de instrucciones de otras estructuras de selección. Esto permite crear una disposición de bifurcaciones en el flujo de control para dotar al programa de un comportamiento complejo, para comprobar si se cumplen diferentes condiciones de acuerdo con cada situación.

### 2.4.1 Ejemplo: descuento máximo y control de errores

Para ver un ejemplo en el que resulta útil la combinación de estructuras de selección, imagine que desea realizar un par de modificaciones en el ejemplo del cálculo de un descuento. Por un lado, que exista un valor máximo de descuento, de modo que si por algún motivo corresponde realizar un descuento por encima de dicho valor, sólo se aplique el máximo establecido. Por otra parte, estaría bien corregir un comportamiento algo raro que ahora mismo tiene el programa: que es capaz de aceptar precios negativos. En este caso, dado que es evidente que el usuario se ha equivocado, estaría bien avisarle con algún mensaje.

Ahora el programa debería hacer lo siguiente:

1. Decidir cuál es el valor mínimo para optar al descuento, cuánto se descontará y el valor máximo posible.
2. Pedir que se introduzca el precio inicial, en euros, por el teclado.
3. Llegir-lo.
4. Comprobar que el precio es correcto y no es negativo:
  - (a) Si se cumple, ver si el precio introducido es igual o superior al valor mínimo para optar al descuento:
    - i. Si es así, calcular el descuento.
    - ii. Comprobar si el descuento supera el máximo permisible:
      - A. Si es así, el descuento se reduce al máximo permisible.
    - iii. Aplicar descuento sobre el precio inicial.
  - (b) Mostrar el precio final.
  - (c) Si el precio era negativo, mostrar un mensaje de error.

Así pues, aparecen dos nuevas condiciones. Antes de hacer nada hay que ver si el precio es positivo y, inmediatamente después de calcular el descuento, ver si se cumple la nueva condición de si el valor es superior al máximo o no. La particularidad es que ahora las diferentes condiciones del programa sólo se comprueban si las anteriores se van cumpliendo, una detrás de otra.

El siguiente código hace uso de una combinación de estructuras de selección para llevar a cabo esta modificación. Compílelo y ejecútelo. Mediante comentarios se asocia cada instrucción en cada uno de los pasos descritos.

```
1 import java.util.Scanner;
2 //Un programa que calcula descuentos.
3 clase pública DescmpteControlErrors {
4     //NO 1
5     //Se realiza un descuento del 8%.
6     public static final float DESCUENTO = 8;
7     //Se hace descuento por compras de 100 euros o más.
8     public static final float COMPRA_MIN = 100;
9     //Valor del descuento máximo: 10 euros.
10    public static final float DESC_MAXIM = 10;
11    public static void main (String[] args) {
12        Scanner lector = new Scanner(System.in);
13        //PASOS 2 y 3
14        System.out.print("¿Cuál es el precio del producto, en euros? ");
15        float precio = lector.nextFloat();
16        lector.nextLine();
17        //PAS 4. ¿El precio es positivo?
18        if (precio > 0) {
19            //YO NO
20            if (precio >= COMPRA_MIN) {
21                //NO es un
22                float descuentoFet = precio * DESCUENTO / 100;
23                //PAS b. Si el descuento supera el máximo, es necesario reducirlo.
24                if (descmpteFet > DESC_MAXIM) {
25                    //PAS alfa. Reducir descuento
26                    descuentoFet = DESC_MAXIM;
27                }
28                //NO c
29                precio = precio - descuentoFet;
30            }
31            //PAS II. Se muestra el precio final.
32            System.out.println("El precio final por pagar es de " + precio + " euros.");
33
34            //PAS III. El precio era negativo. Es necesario avisar al usuario.
35            System.out.println("Precio incorrecto. Es negativo.");
36        }
37    }
38 }
```

Recuerde que en Java cada  
El bloque se identifica como  
entre claus: {...}.

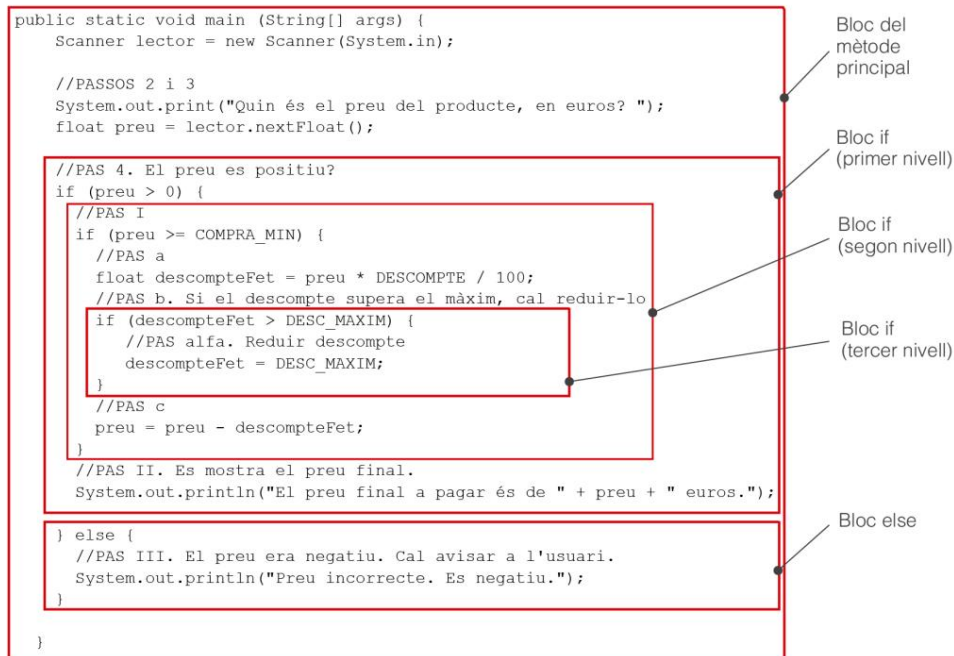
La figura 2.9 muestra un esquema de cómo quedan distribuidos los bloques de instrucciones dentro del programa y de cómo se estructuran jerárquicamente en niveles de profundidad. Como puede apreciarse, cuando se combinan estructuras de control es especialmente importante sangrar el código fuente para hacerlo inteligible y usar siempre claves para poder distinguir claramente dónde comienza y dónde termina cada bloque diferente.

Uno de los aspectos más importantes de este ejemplo es que el programa debe ser capaz de comprobar si los datos que ha introducido el usuario cumplen ciertos condiciones y así avisarle de posibles errores. Muchas veces, para conseguirlo es necesario combinar diferentes estructuras de selección, por lo que sólo se sigue por el camino que procesa los datos si se puede garantizar que son correctos. De hecho, tanto en el ejemplo de adivinar un número como en el de transformar las notas también valdría la pena realizar esta comprobación, y ver si el valor introducido está realmente entre los valores permisibles (0 y 10).



Las estructuras de selección son fundamentales para poder comprobar si los datos que introduce el usuario son correctos antes de procesarlos.

Figura 2. 9. Bloques de instrucciones en una combinación de estructuras de selección



Reto 3: modifique los ejemplos de adivinar (Adivina) un número y transformar una nota numérica en textual (Evaluación Simplificado) para que comprueben que el valor que ha introducido el usuario se encuentra dentro del rango de valores correcto (entre 1 y 10). En algún caso, puede que no sea necesario combinar varias estructuras de selección...

#### 2.4.2 Múltiples bloques de instrucciones y ámbito de variables

En el momento en que el código fuente de un programa se organiza en diferentes bloques de instrucciones, unos dentro de otros, debido a la aparición de estructuras de control, hay que tener cuidado de respetar el ámbito de la declaración de una variable. La mejor manera de ver qué puede ocurrir si no se cuida de esto es siguiendo un ejemplo concreto.

Recuerde que cuando se dice "usar una variable" se quiere decir en realidad "usar el identificador asociado a una variable".

Suponga que ahora desea modificar el programa anterior para que, además del precio final, también muestre exactamente cuántos euros se han descontado sobre el precio inicial. Como se dispone de la variable descuento Hecho ya declarada, donde se almacena precisamente el valor exacto del descuento, sólo debería ser cuestión de consultar su valor y mostrarlo por pantalla, tal y como ya se hace con el precio. Por tanto, de entrada, una posible modificación en el código original sería la que se muestra a continuación. Intégrela en el programa original y intente si funciona o no.

```
1 ...
2     if (precio > 0) {
3         //YO NO
4         if (precio >= COMPRA_MIN) {
5             float descuentoFet = precio * DESCUENTO / 100;
6             if (descuentoFet > DESC_MAXIM) {
7                 descuentoFet = DESC_MAXIM;
8             }
9             precio = precio - descuentoFet;
10        }
11        System.out.println("El precio final por pagar es de " + precio + //Modificación. Ahora también      " euros.");
12        mostramos el descuento... ¿o no?
13        System.out.println("Se ha realizado un descuento de "          + descuentoFet +      " euros."
14        );
15    } demás {
16        System.out.println("Precio incorrecto. Es negativo.");
17    }
18    ...
```

Desafortunadamente, si intenta compilar el programa modificado con este nuevo código, habrá un error en la nueva línea en la que se intenta mostrar el valor del descuento. Éste le informa que el identificador descuentoFet no ha sido declarado (cannot find symbol) y, por tanto, no puede ser usado. Pero apenas unas líneas antes, en la primera instrucción del `if`, existe la declaración. ¿Cómo es posible? La respuesta es que, ahora que ya hay más de un bloque de instrucciones en el código fuente del programa, no se ha respetado su ámbito. Recuerde que la declaración de una variable sólo tiene validez desde que se declara hasta el delimitador de final del bloque en el que se ha declarado (en este caso, la clave `}`).

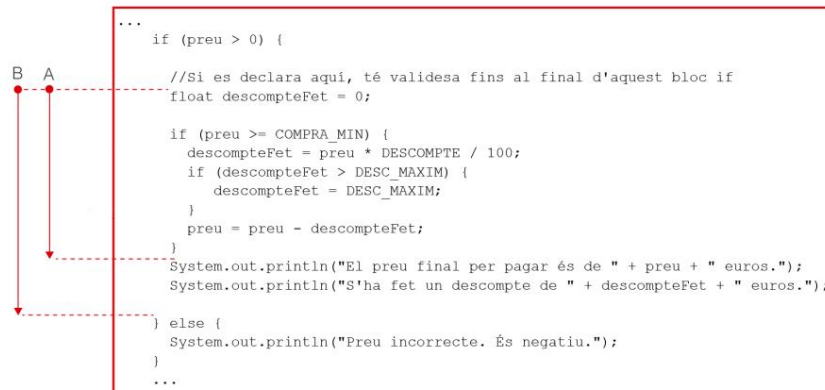
Esta condición se cumple para la variable `precio`, ya que ha sido declarada en el bloque de código del programa principal, y por tanto puede ser usada hasta la clave que cierra el método principal (es decir, en todo el programa). Ahora bien, la variable `descuentoFet` ha sido declarada en el bloque `if` y, por tanto, deja de tener validez al terminar el bloque. Como se consulta más allá de este bloque, es como si no se hubiera declarado nunca.

Para solucionar este problema, la declaración debe llevarse con anterioridad a intentar mostrarla por pantalla y, a la vez, en el mismo bloque de instrucciones, no dentro del bloque `if`. Esto podría hacerse como se muestra a continuación. Incorpora el cambio. Vuelve al código original y comprueba si ahora funciona.

```
1 ...
2     if (precio > 0) {
3         //Si se declara aquí, tiene validez hasta el final de este bloque if.
4         float descuentoFet = 0;
5         if (precio >= COMPRA_MIN) {
6             descuentoFet = precio * DESCUENTO / 100;
7             if (descuentoFet > DESC_MAXIM) {
8                 descuentoFet = DESC_MAXIM;
9             }
10        }
11        precio = precio - descuentoFet;
12    }
13    System.out.println("El precio final por pagar es de " + precio +
14    System.out.println("Se ha realizado un descuento de " + descuentoFet + " euros."
15    );
16
17    } demás {
18        System.out.println("Precio incorrecto. Es negativo.");
19    }
20    ...
```

La figura 2.10 muestra cuál era el ámbito inicial de la variable `descuentoFet` en el programa original (A) y cuál es el que resulta de la modificación de la línea donde se declara (B) para solucionar el error.

Figura 2. 10. Ámbitos de la variable "discountFet"



Una buena pregunta que se puede hacer es qué sentido tiene entonces declarar variables dentro de bloques de código que no sean el método principal. La respuesta es que, a nivel de funcionamiento del programa, no hay ninguna diferencia. Se podrían declarar todas al inicio del programa principal, para garantizar que nunca se encuentre fuera de su ámbito, y todo funcionaría igual. Ahora bien, declarar variables justo en el momento en que se usan tiene sentido en orden a la legibilidad del programa. Si declara una variable cerca del código donde se utiliza, es más sencillo para alguien que lea el código fuente establecer su valor sin tener que repasar todo el código desde el principio. Esto es especialmente útil en programas largos.

## 2.5 El conmutador: la sentencia "switch"

Hay una estructura de selección algo especial, por lo que se ha dejado para el final. Lo que la hace especial es que no se basa en evaluar una condición lógica compuesta por una expresión booleana, sino que establece el flujo de control a partir de la evaluación de una expresión de tipo entero o carácter (pero nunca real).

La sentencia `switch` enumera, uno por uno, un conjunto de valores discretos que se quieren tratar, y asigna las instrucciones a ejecutar si la expresión evalúa en cada valor diferente. Por último, especifica qué hacer si la expresión no ha evaluado en ninguno de los valores enumerados. Es como un conmutador o una palanca de cambios, en la que se asigna un código para cada valor posible a tratar. Este comportamiento sería equivalente a una estructura de selección múltiple en la que, implícitamente, todas las condiciones son comparar si una expresión es igual a cierto valor. Cada rama controlaría un valor distinto. El caso final es equivalente al `else`.

De hecho, en la mayoría de casos, esta sentencia no aporta nada desde el punto de vista del flujo de control que no pueda realizarse con una selección múltiple. Pero es muy útil para mejorar la legibilidad del código o facilitar la generación del código del programa. Ahora bien, sí hay un pequeño detalle en el que ésta



Una sentencia "switch" es un selector entre distintos valores.

sentencia aporta algo que el resto de estructuras de selección no pueden hacer directamente: ejecutar de forma consecutiva más de un bloque de código relativo a diferentes condiciones.

### 2.5.1 Sintaxis y comportamiento

La sintaxis de la sentencia switch se ajusta al formato descrito a continuación. Nuevamente, se basa en una palabra clave seguida de una condición entre paréntesis y un blog de código entre claves. Ahora bien, la diferencia respecto a las sentencias vistas hasta ahora es que los conjuntos de instrucciones asignados a cada caso independiente no se distinguen entre sí por claves. Éstos se van enumerando como un conjunto de apartados etiquetados para la palabra clave, seguida del valor a tratar y dos puntos. De manera opcional, se puede poner una instrucción especial que sirve de delimitador de final de apartado, llamada break. Al final de todos los apartados se pone un de especial, llamado default, que nunca debe terminar en break.

```
1 switch(expresión de tipo entero) {  
2     caso valor1:  
3         instrucciones si la expresión evalúa a valor1  
4         (opcionalmente) break;  
5     caso valor2:  
6         instrucciones si la expresión evalúa a valor2  
7         (opcionalmente) break;  
8     ...  
9     caso valorN:  
10        instrucciones si la expresión evalúa a valorN  
11        (opcionalmente) break;  
12    por defecto:  
13        instrucciones si la expresión evalúa a algún valor que no es valor1...valorN  
14 }
```

cambiar

El nombre de la sentencia switch  
vol dir básicament: "commuta  
entre estas acciones de acuerdo  
con un valor seleccionado".

Al ejecutar la sentencia switch se evalúa la expresión entre paréntesis e inmediatamente se ejecuta el conjunto de instrucciones asignado a ese valor entre los diferentes apartados case. Si no hay ninguno con este valor entre los disponibles, entonces se ejecuta el apartado etiquetado como default (por defecto).

Cabe decir que los valores numéricos no deben seguir ningún orden en concreto para definir cada apartado. Si bien puede ser más pulido enumerarlos por orden creciente, esto no tiene ningún efecto sobre el comportamiento del programa.

La particularidad especial de esta sentencia la tenemos en el uso de la instrucción break. Ésta indica qué hacer una vez ejecutadas las instrucciones del apartado.

Si aparece la instrucción, el programa se salta el resto de apartados y continúa con la instrucción posterior a la sentencia switch (después de la clave que cierra el blog).

En este aspecto, si todos los apartados terminan en break, el funcionamiento de switch es equivalente a hacer:

```
1 if (expresión == valor 2) {  
2     instrucciones si la expresión evalúa a valor1  
3 } else if (expresión == valor2) {  
4     instrucciones si la expresión evalúa a valor2  
5 ...  
6 } else if (expresión == valorN) {
```

```

7  instrucciones si la expresión evalúa a valorN 8 } else { instrucciones
si la expresión
9  evalúa a algún valor que no es valor1...valorN
10 }

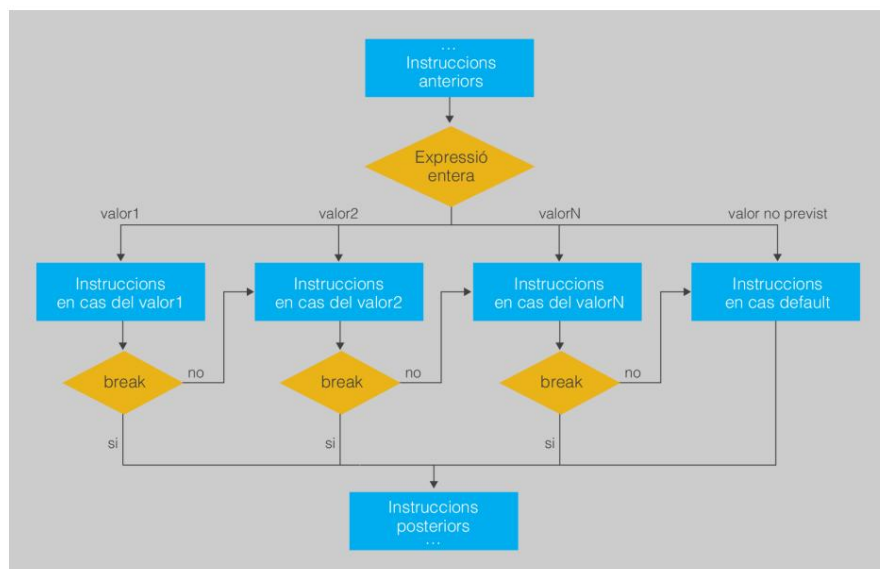
```

Ahora bien, si alguno de los apartados no termina en break, al ejecutar la última instrucción, en lugar de saltar el resto de apartados, lo que se hace es seguir ejecutando las instrucciones del apartado que viene inmediatamente después. Así irá haciendo, apartado por apartado, hasta encontrar alguno que acabe en break. La figura 2.11 muestra ese comportamiento tan particular.

romper

La palabra clave break es como decir "rompe, sale fuera de esta sentencia switch".

Figura 2.1 1. Diagrama de flujo de control de una instrucción "conmutadora"



## 2.5.2 Ejemplo simple: diferentes opciones en un menú

La mejor manera de entender cómo se comporta la sentencia switch es viendo un par de ejemplos. Para empezar, se mostrará el caso más sencillo, en el que todos los apartados tienen una instrucción break al final y, por tanto, es como tener una selección múltiple en la que cada caso corresponde directamente a una condición claramente diferenciada con su propio bloque de instrucciones independiente de los demás.

Un uso muy típico de la sentencia switch es para generar de forma sencilla menús en los que el usuario puede elegir entre diferentes opciones. Según la opción escogida, se ejecutan directamente las instrucciones asociadas a cada una. Suponga un programa en el que, a partir de dos números enteros, se pide qué operación se quiere realizar entre ellos: sumar, restar, multiplicar o dividir.

El programa debería hacer lo siguiente:

1. Preguntar els dos nombres enters.
2. Llegir-los.
3. Mostrar un menú con las cuatro opciones posibles, enumeradas de 1 a 4.
4. Leer la opción elegida.

## 5. Según la opción:

- (a) Si es 1 se suman los dos enteros.
- (b) Si es 2 se restan.
- (c) Si es 3 se multiplican.
- (d) Si es 4 se dividen.
- (e) Si es otra es necesario informar que la opción es incorrecta.

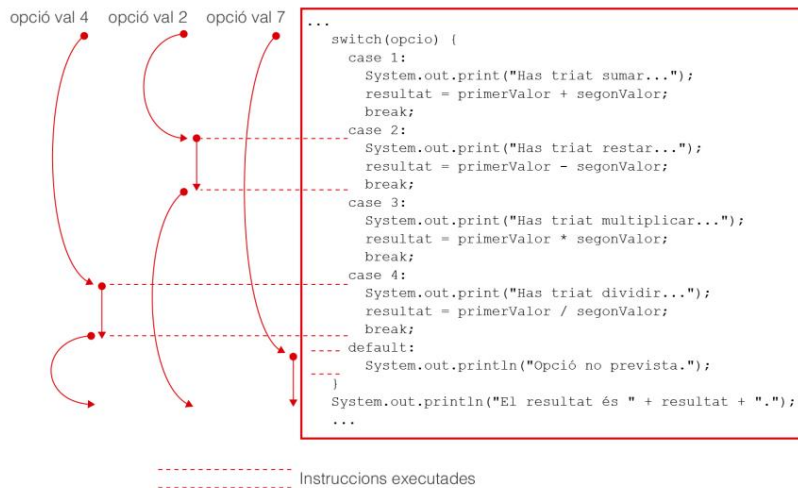
## 6. Finalmente se muestra el resultado por pantalla.

El código para ello mediante la sentencia switch podría ser el siguiente.  
Compílole, ejecútelo y observe su comportamiento.

```
1 import java.util.Scanner;
2 //Un programa que da diferentes opciones de cálculo, usando la sentencia switch.
3 public class Opciones {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //NO 1 i 2
7         System.out.print("Introduce un valor enter: ");
8         int primerValor = lector.nextInt();
9         lector.nextLine();
10        System.out.print("Introduce otro: ");
11        int segonValor = lector.nextInt();
12        lector.nextLine();
13        //NO 3 i 4
14        System.out.println("¿Qué operación quieres hacer? ");
15        System.out.println("[1] Sumar");
16        System.out.println("[2] Reiniciar");
17        System.out.println("[3] Multiplicar");
18        System.out.println("[4] Dividir");
19        System.out.print("Selecciona la opción [1-4]: ");
20        int opcio = lector.nextInt();
21        lector.nextLine();
22        entero resultado = 0;
23        //NO 5
24        interruptor(opción) {
25            //YO NO
26            caso 1:
27                System.out.print("Has triat sumar...");
28                resultado = primerValor + segundoValor;
29                romper;
30            //NO II
31            caso 2:
32                System.out.print("Se ha reiniciado triat...");
33                resultado = primerValor - segundoValor;
34                romper;
35            //PASO III
36            caso 3:
37                System.out.print("Tiene multiplicador triat...");
38                resultado = primerValor * segundoValor;
39                romper;
40            //NO ES IV
41            caso 4:
42                System.out.print("Tiene triat dividir...");
43                resultado = primerValor / segundoValor;
44                romper;
45            //NO V
46            por defecto:
47                System.out.println("Opción no prevista.");
48        }
49        //NO 6
50        System.out.println("El resultado es" + resultado + ".");
51    }
52 }
```

La figura 2.12 muestra un esquema de las instrucciones ejecutadas según algunos valores de ejemplo. En lugar de usar un diagrama de flujo de control, se usa directamente el código como referencia para hacer la relación más esclarecedora. Como puede verse, la opción default es especialmente útil para controlar valores de entrada no válidos.

Figura 2.1 2. Ejemplo de flujo de control del programa de opciones de cálculo



### 2.5.3 Ejemplo con propagación de instrucciones: los días de un mes

Es el momento de ver qué ocurre si faltan instrucciones break en algunos apartados.

Lo primero que puede probar es quitar los break del ejemplo anterior. Verá qué pasa. Cada vez que elija una opción se ejecutará aquella opción y todas las posteriores (lo podrá ver, ya que se irá mostrando por pantalla el mensaje para cada operación). Por tanto, el único resultado válido será el último y siempre estará dividiendo.

Otro ejemplo distinto bastante típico es el de calcular cuántos días tiene un mes. A grandes rasgos, el programa haría lo siguiente:

1. Preguntar el número de mes (un valor de tipus enter).
2. Llegir-lo.
3. Según el valor del mes:
  - (a) Si es 2 hay que decir que el número de días es 28 o 29.
  - (b) Si es 4, 6, 9 o 11 hay que decir que el número de días es 30.
  - (c) Si es 1, 3, 5, 7, 8, 10 o 12 hay que decir que el número de días es 31.
  - (d) Si es otro, hay que decir que se ha introducido un número de mes incorrecte.

Ahora bien, se jugará ahora con la propagación de instrucciones cuando falta el break para establecer dentro del código conjuntos de valores que dan el mismo resultado. Al elegir un valor, se ejecutan las instrucciones del caso asociado a ese valor y, mientras no encuentre

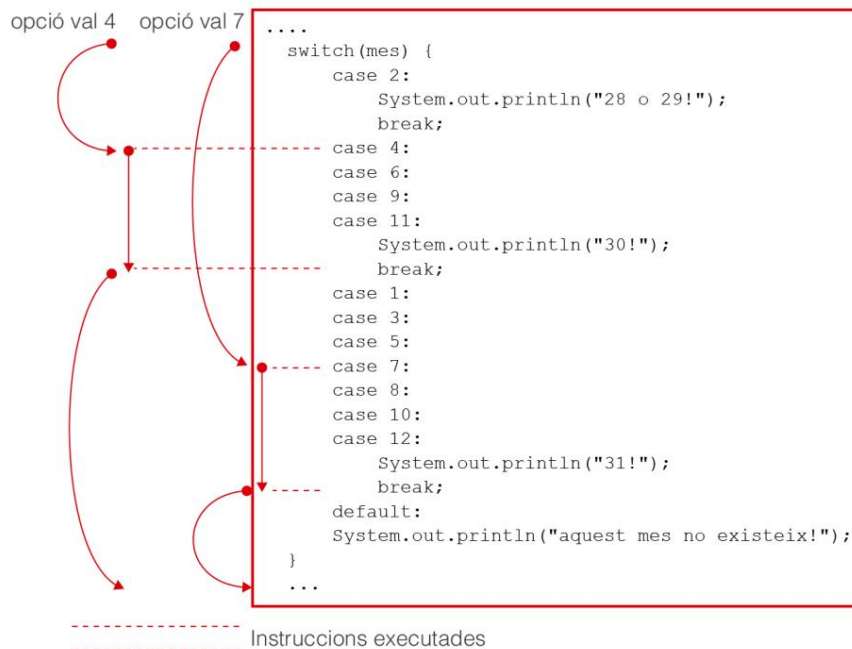
un break, sigue ejecutando el resto de casos consecutivamente, uno detrás de otro. Si un caso está vacío, no hace nada y salta directamente a lo siguiente. Como se puede ver en el ejemplo, no es necesario que los valores para cada caso estén ordenados de acuerdo con ningún orden numérico concreto. Una vez más, el caso default sirve para controlar valores fuera de rango. Compruebe que el programa hace lo que se espera en su entorno de desarrollo.

```
1 import java.util.Scanner;
2
3 //A ver cuántos días tiene un mes...
4 clase pública DiesDelMes {
5
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8
9         //NO 1 i 2
10        System.out.print("¿Qué número de mes quieres analizar [1-12]? ");
11        int mes = lector.nextInt();
12        lector.nextLine();
13
14        //NO 3
15        System.out.print("Los días de este mes serán...");
16        interruptor(mes) {
17            //YO NO
18            caso 2:
19                System.out.println("28 o 29!");
20                romper;
21            //NO II
22            caso 4:
23            caso 6:
24            caso 9:
25            caso 11:
26                System.out.println("30!");
27                romper;
28            //PASO III
29            caso 1:
30            caso 3:
31            caso 5:
32            caso 7:
33            caso 8:
34            caso 10:
35            caso 12:
36                System.out.println("31!");
37                romper;
38            por defecto:
39                System.out.println("¡Este mes no existe!");
40        }
41
42    }
43
44 }
```

La figura 2.13 muestra un par de ejemplos de cuál es el flujo de control de la sentencia switch para ciertos valores. Tenga en cuenta que el programa comienza por el valor que coincide con el resultado de evaluar la expresión y que, una vez allí, va avanzando de forma secuencial por todos los casos, haya lo que haya, hasta encontrar una sentencia break.



Figura 2. 1 3. Ejemplo de flujo de control para el programa de los días del mes. Una sentencia "switch" con casos sin "break"



Por último, cabe decir que, como ya se ha mencionado, no hay nada que haga la sentencia switch que no pueda realizar una selección múltiple. La elección de switch suele hacerse exclusivamente por motivos de claridad del código. Compare el código anterior con el siguiente, equivalente. A continuación, compruebe que hace lo mismo.

```

1 import java.util.Scanner;
2
3 //A ver cuántos días tiene un mes...
4 clase pública DiesDelMesIf {
5
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8
9         System.out.print("¿Qué número de mes quieres analizar [1-12]? ");
10        int mes = lector.nextInt();
11        lector.nextLine();
12
13        System.out.print("Los días de este mes serán...");
14        si (mes == 2) {
15            System.out.println("28 o 29!");
16        } else if ((mes == 4)||(mes == 6)||(mes == 9)||(mes == 11)) {
17            System.out.println("30!");
18        } else if ((mes == 1)||(mes == 3)||(mes == 5)||(mes == 7)||(mes == 8)||(mes
19            == 10)||(nosotros == 12)) {
20            System.out.println("31!");
21        } demás {
22            System.out.println("¡Este mes no existe!");
23        }
24    }
25 }

```

Decidir cuál es más práctico es su decisión. De todas formas, no suele ser buena idea tener expresiones tan largas que se salgan del campo de visión en el editor empleado.

## 2.6 Control de errores en la entrada básica mediante estructuras de selección

Las estructuras de selección son especialmente útiles como mecanismo para controlar si los datos que ha introducido un usuario son correctos antes de usarlos dentro del programa. Una vez sabéis cómo funcionan, es un buen momento para ampliar las explicaciones sobre la entrada básica de datos por teclado, de forma que sea posible controlar si el valor que se ha introducido encaja con el tipo de dato esperado para cada instrucción de lectura.

Si se parte que la extensión para leer datos de teclado se ha inicializado anteriormente Como siempre:

```
1 Scanner lector = new Scanner(System.in);
```

Antes de leer cualquier dato introducido por el teclado, es posible usar las instrucciones de la tabla 2.2 para establecer si el dato que apenas se ha escrito por el teclado es de un tipo concreto o no. Cuando se usan, el usuario puede introducir datos por el teclado (el programa se detiene) y se evalúa si lo que ha escrito pertenece a un tipo de dato o un otro. Estas instrucciones se consideran expresiones booleanas que evalúan cómo a true si el tipo concuerda y como false si no es el caso.

Una vez utilizada esta instrucción, sólo se podrán leer los datos correctamente usando la instrucción `lector.next...` correspondiente al mismo tipo si ha evaluado como true. Si se intenta después de que haya evaluado false, el error de ejecución está garantizado.

Tabla 2. 2. Instrucciones para la lectura de datos por el teclado

| Instrucción                          | Tipo de datos controlar |
|--------------------------------------|-------------------------|
| <code>lector.hasNextByte()</code>    | byte                    |
| <code>lector.hasNextShort()</code>   | corto                   |
| <code>lector.hasNextInt()</code>     | entero                  |
| <code>lector.hasNextLong()</code>    | largo                   |
| <code>lector.hasNextFloat()</code>   | flotar                  |
| <code>lector.hasNextDouble()</code>  | doble                   |
| <code>lector.hasNextBoolean()</code> | booleano                |

La instrucción asociada a una cadena de texto no existe, ya que cualquier dato escrita por el teclado siempre se puede interpretar como texto. Por tanto, la lectura nunca puede dar error.

Por ejemplo, el programa pensado para adivinar un número podría hacerse tal y como muestra el código siguiente. Presta atención al comportamiento de las combinaciones estructuras de selección. Fíjese que hay tres caminos disjuntos posibles fruto de este hecho: si el dato escrito no es un entero, si lo es pero no acierta el número, y finalmente, si lo es y sí acierta. Compílelo y ejecútelo por

ver que, efectivamente, ahora se puede detectar que los datos introducidos no son del tipo correcto.

```
1 import java.util.Scanner;
2 //Un programa en el que es necesario adivinar un número.
3 public class AdivinaControlErrorsEntrada {
4     //NO 1
5     //El número a adivinar será el 4.
6     public static final int VALOR_SECRET = 4;
7     public static void main (String[] args) {
8         Scanner lector = new Scanner(System.in);
9         System.out.println("Empezamos el juego.");
10        System.out.print("Endevina el valor enter, entre 0 i 10: ");
11        boolean tipusCorrecto = lector.hasNextInt();
12        si (tipusCorrecto) {
13            //Se ha escrito un entero correctamente. Ya se puede leer.
14            int uservalue = reader.nextInt();
15            lector.nextLine();
16            Si (SECRET_VALUE == uservalue) {
17                System.out.println("¡Exacto! Era " + VALOR_SECRETO + ".");
18            }
19            System.out.println("¡Es un error!");
20        }
21        System.out.println("Hemos terminado el juego.");
22    } demás {
23        //No se ha escrito un entero.
24        System.out.println("El valor introducido no es un entero.");
25    }
26 }
27 }
```

Reto 4: aplique el mismo tipo de control sobre los datos de la entrada a el ejemplo del cálculo del descuento.

## 2.7 Soluciones de los retos propuestos

### Reto 1:

```
1 import java.util.Scanner;
2 public class Penalizacio {
3     public static final float PENALIZACION = 2;
4     //Se hace descuento por compras de 100 euros o más.
5     public static final float COMPRA_MIN = 30;
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8         System.out.print("¿Cuál es el precio del producto en euros? ");
9         float precio = lector.nextFloat();
10        lector.nextLine();
11        if (precio < COMPRA_MIN) {
12            //YO NO
13            precio = precio + PENALIZACIÓN;
14        }
15        System.out.println("El precio final por pagar es de " + precio + " euros.");
16    }
17 }
```

### Reto 2:

```
1 import java.util.Scanner;
2 clase pública DosSecretos {
3     //NO 1
4     //Los números para adivinar serán el 4 y el 7.
5     public static final int VALOR_SECRET_UN = 4;
6     public static final int OTRO_VALOR_DOS = 7;
7     public static void main (String[] args) {
8         Scanner lector = new Scanner(System.in);
9         //NO 2 i 3
10        System.out.println("Empezamos el juego.");
11        System.out.print("Endevina el valor enter, entre 0 i 10: ");
12        int uservalue = reader.nextInt();
13        lector.nextLine();
14        //NO 4
15        //Estructura de selección doble.
16        //O adivina o falla.
17        Si ((VALOR_SECRET_UNO == valor_usuario)|| (OTRO_VALOR_DOS == valor_usuario)) {
18            //YO NO
19            //Si la expresión evalúa true, ejecuta el bloque en el if.
20            System.out.println("¡Exacto! Era " + uservalue + ".");
21        } demás {
22            //NO II
23            //Si la expresión evalúa false, ejecuta el bloque dentro del les.
24            System.out.println("¡Es un error!");
25        }
26        System.out.println("Hemos terminado el juego.");
27    }
28 }
```

## Reto 3:

```
1 import java.util.Scanner;
2 public class AdivinaControlValores {
3     public static final int VALOR_SECRET = 4;
4     public static void main(String[] args) {
5         Scanner lector = new Scanner(System.in);
6         System.out.println("Empezamos el juego.");
7         System.out.print("Endevina el valor enter, entre 0 i 10: ");
8         int uservalue = reader.nextInt();
9         lector.nextLine();
10        Si ((uservalue < 0) || (uservalue > 10)){
11            System.out.println("Error: el valor debe estar entre 0 y 10.");
12        }demás{
13            Si (SECRET_VALUE == uservalue) {
14                System.out.println("¡Exacto! Era " + VALOR_SECRETO + ".");
15            }
16            System.out.println("¡Es un error!");
17        }
18        System.out.println("Hemos terminado el juego.");
19    }
20 }
21 }
```

```
1 import java.util.Scanner;
2 public class EvaluacioControlValors {
3     public static void main(String[] args) {
4         Scanner lector = new Scanner(System.in);
5         System.out.print("Quina nota has tret? ");
6         float nota = lector.nextFloat();
7         lector.nextLine();
8         si ((nota >= 9) && (nota <= 10)) {
9             System.out.println("Tu nota final es Excelente.");
10        } else if ((nota >= 6.5) && (nota < 9)) {
11            System.out.println("Tu nota final es Notable.");
12        } else if ((nota >= 5) && (nota < 6.5)) {
13            System.out.println("Tu nota final es Aprobado.");
14        } else if ((nota >= 0) && (nota < 5)) {
15            System.out.println("Tu nota final es Suspendido.");
16        } demás {
17            System.out.println("¡El valor escrito no está entre 0 y 10!");
18        }
19    }
20 }
```

## Reto 4:

```
1 import java.util.Scanner;
2 public class DescompteControlErrorsEntrada {
3     //Se realiza un descuento del 8%.
4     public static final float DESCUENTO = 8;
5     //Se hace descuento por compras de 100 euros o más.
6     public static final float COMPRA_MIN = 100;
7     //Valor del descuento máximo: 10 euros.
8     public static final float DESC_MAXIM = 10;
9     public static void main (String[] args) {
10         Scanner lector = new Scanner(System.in);
11         System.out.print("¿Cuál es el precio del producto en euros? ");
12         booleano tipusCorrecto = lector.hasNextFloat();
13         //¿El tipo es correcto?
14         si (tipusCorrecto) {
15             float precio = lector.nextFloat();
16             lector.nextLine();
17             if (precio > 0) {
18                 if (precio >= COMPRA_MIN) {
19                     float descuentoFet = precio * DESCUENTO / 100;
20                     if (descuentoFet > DESC_MAXIM) {
21                         descuentoFet = DESC_MAXIM;
22                     }
23                     precio = precio - descuentoFet;
24                 }
25                 System.out.println("El precio final por pagar es de " + precio + " euros."
26                                     );
27             } demás {
28                 System.out.println("Precio incorrecto. Es negativo.");
29             }
30         } demás {
31             //No se ha escrito un entero.
32             System.out.println("El valor introducido no es un entero.");
33         }
34 }
```

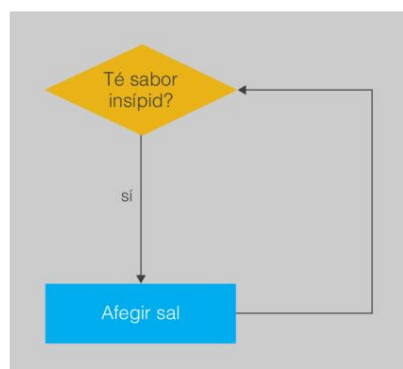
### 3. Estructuras de repetición

Hasta el momento, todos los programas que ha estudiado, ya fueran más complejos o más sencillos, tenían una característica en común. Su flujo de control avanzaba inexorablemente hacia la instrucción final del método principal sin posibilidad de retroceder. Sea cual sea el camino alternativo por el que se ha llegado, una vez una instrucción ha sido ejecutada, ya no se volverá a ejecutar nunca más. Este hecho contrasta con el comportamiento que se puede observar en muchas actividades que hacemos a diario, como por ejemplo, cuando ponemos sal en la comida que cocinamos. Podemos repetir varias veces el proceso de poner sal y probarlo hasta que el gusto sea el que queremos (y entonces ya dejamos de poner sal y probar). Este proceso se representa como un diagrama de flujo de control en la figura 3.1.



Una estructura de repetición permite ejecutar las mismas instrucciones en varias ocasiones. Imagen de Wikimedia Commons

Figura 3. 1. Una acción se repite mientras se cumpla una condición



Esto también ocurre en programas de su ordenador, en los que se pueden repetir acciones sin problemas: abrir y guardar diferentes archivos, conectarse a diferentes páginas web, etc. Aquí es donde entra en juego el siguiente tipo de estructura de control, también muy importante dentro del código de un programa.

Las estructuras de repetición o iterativas permiten repetir una misma secuencia de instrucciones varias veces, mientras se cumpla cierta condición.

En su aspecto general, tienen muy parecido a una estructura de selección.

Hay una sentencia especial que hay que escribir en el código fuente, unida a una condición lógica y un bloque de código (en este caso, siempre será sólo uno). Pero en ese caso, mientras la condición lógica sea cierta, toda la secuencia de instrucciones se va ejecutando repetidamente. En el momento en que se deja de cumplir la condición, se deja de ejecutar el bloque de código y ya se sigue con la instrucción que existe después de la sentencia de la estructura de repetición.

Llamamos bucle o ciclo al conjunto de instrucciones que debe repetirse un cierto número de veces, y llamamos iteración a cada ejecución individual del bucle.

Como ocurre con las estructuras de selección, hay diferentes tipos de sentencias, cada una con sus particularidades. Normalmente, la diferencia principal está vinculada al momento en que se evalúa la condición para ver si es necesario volver a repetir el bloque de instrucciones o no. A lo largo de este apartado, las irás viendo con detalle.

### 3.1 Control de las estructuras repetitivas

Como ocurría con las estructuras de selección, las estructuras de repetición no tienen sentido a menos que la condición lógica depende de alguna variable que pueda ver modificado su valor para diferentes ejecuciones. De lo contrario, la condición siempre valdrá lo mismo para cualquier ejecución posible y utilizar una estructura de control no será muy útil. En este caso, si la condición siempre es false, nunca se ejecuta el bucle, por lo que es código inútil. Sin embargo, para las estructuras de repetición, si la condición siempre es true el problema es mucho más grave. Como absolutamente siempre que se evalúa si es necesario realizar una nueva iteración, la condición se cumple, el bucle no se deja nunca de repetir. ¡El programa nunca acabará!

Un bucle infinito es una secuencia de instrucciones dentro de un programa que itera indefinidamente, normalmente porque se espera que se alcance una condición que nunca llega a producirse.

Bucle infinito  
Un bucle infinito es un error  
semántico de programación. Si los  
hubiere, el programa compilará  
perfectamente de todos modos.

Cuando esto sucede, el programa no se puede detener de otra forma que no sea cerrándolo directamente desde el sistema operativo (por ejemplo, cerrando la ventana asociada o con alguna secuencia especial de escape del teclado) o usando algún mecanismo de control de la ejecución de su programa que ofrezca el IDE usado.

Teniendo en cuenta el peligro de que un programa se acabe ejecutando indefinidamente, forzosamente dentro de todo bucle deben existir instrucciones que manipulen variables cuyo valor permita controlar la repetición o el final del bucle. Estas variables se llaman variables de control.

Garantizar la correcta asignación de valores de las variables de control de una estructura repetitiva es extremadamente importante. Cuando genere un programa, es imprescindible que el código permita que, en algún momento, la variable cambie de valor, de modo que la condición lógica se deje de cumplir. Si esto no es así, tendrá un bucle infinito.

Normalmente, las variables de control dentro de un bucle pueden englobarse dentro de alguno de estos tipos de comportamiento:

- Contador: una variable de tipo entero que va aumentando o disminuyendo, indicando de forma clara el número de iteraciones a realizar.



- **Acumulador:** una variable en la que se van acumulando directamente los cálculos que se quieren realizar, por lo que al alcanzar cierto valor se considera que ya no es necesario realizar más iteraciones. Si bien se asemejan a los contadores, no son exactamente el mismo.
- **Semáforo:** una variable que sirve como interruptor explícito de si es necesario seguir haciendo iteraciones. Cuando ya no queremos hacer más, el código simplemente se encarga de asignarle el valor específico que servirá para que la condición evalúe false.

---

Los semáforos también se llaman popularmente flags (banderolas de aviso).

---

Evidentemente, una condición lógica puede tomar la forma de una expresión muy compleja, pero para empezar es suficiente con conocer estos tres modelos. En ocasiones, las diferencias pueden ser sutiles, por lo que tampoco hay que preocuparse demasiado si no se tiene claro a qué tipo pertenece exactamente la variable de control. Igualmente, tener presentes estos tres papeles básicos puede ser de ayuda de cara a enfocar la programación de una estructura de selección.

### 3.2 Repetir si se cumple una condición: la sentencia while

La estructura de repetición por antonomasia es la codificada mediante la sentencia while. Ésta existe de una manera u otra en la mayoría de lenguajes de programación. Su particularidad es que prácticamente cualquier código basado en una estructura de repetición puede representarse usando esta sentencia. Con esta ya tendría suficiente para tratar casi cualquier caso posible. Esto sucede con contraposición de las estructuras de selección, en las que cada tipo de sentencia ofrece distintas posibilidades.

La sentencia while permite repetir la ejecución del bucle mientras se verifique la condición lógica. Esta condición se verifica al principio de cada iteración. Si la primera vez, justo cuando se ejecuta la sentencia por primera vez, ya no se cumple, no se ejecuta iteración alguna.

#### 3.2.1 Sintaxis y estructura

Para realizar este tipo de control sobre las iteraciones de un bucle, la sintaxis de esta sentencia en el lenguaje Java es la siguiente:

```
1 while (expresión booleana) {  
2     Instrucciones para ejecutar dentro del bucle  
3 }
```

Como puede ver, su formato es muy parecido a la sentencia if, simplemente cambiando la palabra clave por while. Como ya ocurría con las diferentes sentencias

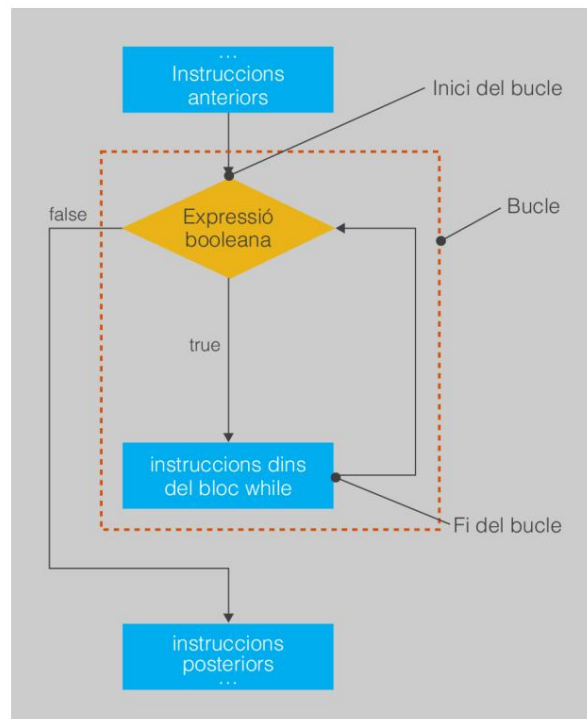
mientras

El nombre de la sentencia while básicamente significa: "Mientras esto se cumpla, haz esto otro..."

dentro de las estructuras de selección, si entre los paréntesis se pone una expresión que no evalúa un resultado de tipo booleano, habrá un error de compilación.

La figura 3.2 muestra un diagrama del flujo de control de esta sentencia, estableciendo el orden bajo el cual se evalúa la condición lógica representada por la expresión booleana de cara a establecer si es necesario ejecutar o no una iteración del bucle.

Figura 3.2. Diagrama de flujo de control de una instrucción "while".



La mejor forma de ver cómo funciona y constatar su utilidad es mediante algunos ejemplos concretos.

### 3.2.2 Ejemplo: ahorrarse de escribir lo mismo muchas veces

La utilidad más directa de una estructura de repetición es que, en caso de que desee ejecutar exactamente las mismas instrucciones muchas veces, no las tenga que escribir un montón de golpes. Por ejemplo, suponga que desea escribir una línea horizontal en la pantalla exactamente con cuarenta caracteres '-'. Evidentemente, nada le impide pulsar exactamente 40 veces la tecla, ni una más ni una menos, y ya está. Pero, ¿también lo haría si es necesario escribir cien, mil o incluso más? Hacerlo con una estructura de repetición es más cómodo.

Para alcanzar este objetivo, es necesario que el bucle itere un cierto número de veces. Para ello, necesitará una variable de control de tipo contador, que sirva para contar cuántas iteraciones hay hechas. O sea, cada vez que se produce una iteración, sumar 1. Una vez que esta variable supera el valor que queremos, ya no es necesario realizar más iteraciones. Por tanto, la condición lógica debe controlar si el valor de la variable es inferior al número de iteraciones a realizar.

Un posible código que ilustra esto sería lo siguiente. Compílelo y ejecútelo por ver qué hace:

```
1 //Un programa que escribe una línea con 100 caracteres '-'.
2 class pública Linia {
3     public static void main (String[] args) {
4         //Inicializamos un contador
5         entero i = 0;
6         //¿Ya hemos hecho esto 100 veces?
7         mientras (i < 100) {
8             System.out.print("-");
9             //Lo hemos hecho una vez, sumamos 1 al contador
10            i = i + 1;
11        }
12        //Forzamos un salto de línea
13        System.out.println();
14    }
15 }
```

Hay un par de cuestiones del código anterior que vale la pena comentar. De una Por otro lado, la nomenclatura de la variable de control de tipo contador. Evidentemente, se puede usar el identificador que se desee, pero es común usar letras individuales (i, j, k...) para poder identificar rápidamente qué variable dentro de un bucle es un contador y facilitar su comprensión. Por otra parte, a la hora de usar contadores, suele empezarse desde el valor 0 en lugar de 1.

Para ver con más detalle qué está pasando en un bucle, en lugar de un esquema basado en el diagrama de flujo de control utilizaremos una tabla. Cada fila corresponderá en una iteración y en cada columna se especificará el valor de las diferentes variables dentro del bucle, por lo que se puede ver cómo evolucionan en cada iteración. Normalmente, se hace énfasis en el valor al inicio del bucle, justo antes de evaluar la condición lógica, y al final del bucle, justo después de la última instrucción, por ver cuáles han sido las modificaciones al ejecutar las instrucciones contenidas. Evidentemente, los valores de las variables al inicio de una iteración deben ser exactamente los mismos que justo al inicio de la siguiente.

La tabla 3.1 muestra un resumen de cómo cambia el valor de la variable de control y de cuál es el resultado de evaluar la condición lógica en cada iteración para el ejemplo actual.

Tabla 3. 1. Evolución del bucle para escribir 100 caracteres "-"

| Iteración | Inicio del bucle | Alambre en bucle      |                         |
|-----------|------------------|-----------------------|-------------------------|
|           | "i" val          | Condición vale        | "i" val                 |
| 1         | 0                | (0 < 100), verdadero  | 1                       |
| 2         | 1                | (1 < 100), verdadero  | 2                       |
| 3         | 2                | (2 < 100), verdadero  | 3                       |
| ...       |                  |                       |                         |
| 99        | 98               | (98 < 100), verdadero | 99                      |
| 100       | 99               | (99 < 100), verdadero | 100                     |
| 101       | 100              | (100 < 100), falso    | Ja hem sortit del bucle |



Un contador controla el número de iteraciones a realizar. Imagen de Wikimedia Commons

Coloquialmente, por decir que ya no es necesario realizar más iteraciones se sol dir que "se surt del "bucles".

Reto 1: en muchos casos, el número de repeticiones no será fijo, sino que podrá depender de otra variable. Modifique el ejemplo para que primero pregunte a el usuario cuántos caracteres '-' quiere escribir por pantalla, y entonces los escriba. Cuando pruebe el programa, no introduzca un número muy alto!

### 3.2.3 Ejemplo: aprovechar un contador

La utilidad de las estructuras de selección va mucho más allá de ser una forma sencilla de ahorrarse escribir el mismo código muchas veces. Con estas se pueden ejecutar instrucciones de modos que no se pueden replicar simplemente haciendo muchas veces “copiar” y “pegar” dentro del código fuente. Y es que, a la hora de la verdad, un contador suele tener un papel más importante que simplemente contar el número de iteraciones que se han realizado. El valor que almacena también se puede utilizar dentro del bucle para realizar otros propósitos.

Por ejemplo suponga que desea imprimir por pantalla todos los valores de la tabla de multiplicar de un número entero cualquiera, desde el 1 al 10. Sin detenernos mucho que pensar, este programa debería hacer:

- 1. Pedir que se introduzca un número por el teclado.
- 2. Llegir-lo.
- 3. Mostrar el resultado de multiplicar el número por 1.
- 4. Mostrar el resultado de multiplicar el número por 2.
- ...
- 12. Mostrar el resultado de multiplicar el número por 10.

De esta descripción se desprende que del paso 3 al 12 se están repitiendo órdenes, que seguramente se pueden reemplazar por una estructura de repetición. Concretamente hay 10 pasos entre los 3 y los 12, ya cada uno lo que se hace es más o menos lo mismo: multiplicar el valor introducido por un valor que cada vez se incrementa en 1. Es es decir, es necesario un contador. Pero este contador ahora, además de garantizar que se hacen 10 iteraciones, también se usa para realizar cálculos.

De acuerdo a esto, un posible código podría ser el siguiente. Pruebe que funciona.

```
1 import java.util.Scanner;
2 //Un programa que muestra la tabla de multiplicar de un número.
3 clase pública TaulaMultiplicar {
4     public static void main (String[] args) {
5         //S'inicialitza la biblioteca.
6         Scanner lector = new Scanner(System.in);
7         //Pregunta el nombre.
8         System.out.print("¿Qué tabla de multiplicar quieres? ");
9         int tabla = lector.nextInt();
10        lector.nextLine();
11        //El contador servirá para realizar cálculos.
12        entero i = 1;
```

```
13     mientras (i <= 10) {
14         int resultado = tabla*i;
15         System.out.println(taula + i = i + 1;      " * " + yo + " = " + resultado);
16     }
17
18     System.out.println("Esta ha sido la tabla del "      + mesa);
19 }
20 }
```

Fíjese que, aunque normalmente en los contadores se empieza a contar por el 0, en este caso hay que empezar por el 1, ya que dentro de los cálculos a realizar, el primero paso requiere que la multiplicación sea por 1. La [tabla 3.2](#) muestra la evolución de las variables en cada iteración, suponiendo que se realice la tabla de multiplicar del 5.

Tabla 3. 2. Evolución del bucle para escribir la tabla de multiplicar del 5

| Iteración | Inicio del bucle | Alambre en bucle     |                          |         |
|-----------|------------------|----------------------|--------------------------|---------|
|           | 'i' val          | Condición vale       | selección de 'resultado' | 'i' val |
| 1         | 1                | (1 ≤ 10), verdadero  | 5*1, 5                   | 2       |
| 2         | 2                | (2 ≤ 10), verdadero  | 5*2, 10                  | 3       |
| 3         | 3                | (3 ≤ 10), verdadero  | 5*3, 15                  | 4       |
| ...       |                  |                      |                          |         |
| 9         | 9                | (9 ≤ 10), verdadero  | 5*9, 45                  | 10      |
| 10        | 10               | (10 ≤ 10), verdadero | 5*10, 50                 | 11      |
| 11        | 11               | (11 ≤ 10), falso     | Ja hem sortit del bucle  |         |

Reto 2: un contador tanto puede empezar a contar desde 0 e ir subiendo, como desde el final e ir disminuyendo como una cuenta atrás. Modifique este programa para que la tabla de multiplicar comience mostrando el valor para 10 y vaya bajando aletas a l'1.

3.2.4 Ejemplo: no siempre se suma u

A la hora de utilizar variables de control con la función de contador, es muy típico que el incremento sea de uno en uno. Esto es lógico, ya que es la forma más simple de contar cuántas iteraciones se han realizado. Para realizar 10 iteraciones, si bien es lo mismo sumar de uno en uno y comparar con 10 o sumar de dos en dos y comparar con 20, es claro que el primer caso es mucho más comprensible. Ahora bien, cuando los contadores también tienen dentro del bucle un papel para realizar ciertas operaciones, hay que tener presente que se pueden incrementar o modificar en cualquier valor.

Por ejemplo suponga un programa que permita sumar todos los valores enteros múltiplos de 3 dentro de un intervalo entre 0 y un valor cualquiera. Una primera aproximación podría ser:

1. Preguntar el límite del intervalo.
2. Llegir-lo.

3. Ir mirando uno por uno todos los valores dentro del intervalo, de 0 al límite estipulado.

(a) Si es múltiplo de tres, se va acumulando en una variable en la que se guarda el resultat final.

(b) Si no lo es, se ignora.

4. Una vez recorrido todo el intervalo (superado el valor límite), se muestra el resultado acumulado.

---

Para ver si un número es múltiplo de 3 es necesario comprobar si su módulo (%) 3 es igual un 0.

---

El código fuente que haría esto podría ser el siguiente. Este código tiene la particularidad –y por eso vale la pena estudiarlo– que combina estructuras de repetición con estructuras de selección. Ahora bien, el formato de la estructura de repetición no es demasiado diferente a los ejemplos anteriores: un contador que va de 0 a un valor concreto para controlar cuántas veces itera el bucle. Para hacerlo más comprensible cuando lo ejecute, cada vez que se encuentra un múltiplo de 3 lo muestra por pantalla.

```
1 import java.util.Scanner;
2 //Vamos a sumar una serie de múltiplos de tres.
3 clase pública SumarMultiplesTres {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //NO 1 i 2
7         System.out.print("¿Hasta qué valor quieres sumar múltiplos de 3? ");
8         int límite = lector.nextInt();
9         lector.nextLine();
10        entero resultado = 0;
11        //PAS 3: Ir mirando los valores uno por uno. Se comienza por el 0.
12        entero i = 0;
13        mientras (i <= límite) {
14            //PAS a: ¿Es múltiplo de tres?
15            si ( (i%3) == 0) {
16                System.out.println("Afegim " resultado + yo +"...");
17                = resultado + i;
18            }
19            //PAS 3: Seguir ir mirando los valores uno por uno.
20            i = i + 1;
21        }
22        System.out.println("El resultado final es" + resultado + ".");
23    }
24 }
```

Ahora bien, existe una manera de simplificar este programa. Sería mucho más sencillo si, en lugar de ir probando uno por uno todos los valores dentro del rango, el contador siempre tenga únicamente valores múltiplos de 3. En este caso, sólo habría que ir sumando los valores sin necesitar ninguna estructura de selección. Partiendo de este hecho, ¿qué valores debería ir tomando el contador? Pues 0, 3, 6, 9, 12, 15, etc. Si pensamos un poco en esta secuencia de números, se ve que es suficiente que el contador se incremente de tres en tres.

Por tanto, el código del bucle podría reemplazarse por lo siguiente:

```
1 ...
2     mientras (i <= límite) {
3         System.out.println("Afegim " resultado + yo +"...");
4         = resultado + i;
5         //Incrementamos de tres en tres.
6         i = i + 3;
7     }
8     ...
```

Una variable de control con el papel de contador se puede modificar de cualquier forma que se considere oportuno. Ahora bien, siempre hay que garantizar que en cada iteración se acerca a la condición lógica false, evitando así un posible bucle infinito.

La tabla 3.3 muestra la evolución de las variables del programa en cada iteración del programa bucle, suponiendo que se quiere hacer cálculo hasta el número 20. 7

iteraciones es que, si ejecuta el programa, se puede ver cómo se ejecuta exactamente 7 veces la instrucción `System.out.println`.

Tabla 3. 3. Evolución del bucle para escribir la suma de los valores enteros múltiplos de 3

| Iteración | Inicio del bucle |                      | Alambre en bucle         |         |
|-----------|------------------|----------------------|--------------------------|---------|
|           | 'i' val          | Condición vale       | selección de 'resultado' | 'i' val |
| 1         | 0                | (0 ≤ 20), verdadero  | 0+0, 0                   | 3       |
| 2         | 3                | (3 ≤ 20), verdadero  | 0+3, 3                   | 6       |
| 3         | 6                | (6 ≤ 20), verdadero  | 3+6, 9                   | 9       |
| 4         | 9                | (9 ≤ 20), verdadero  | 9+9, 18                  | 12      |
| 5         | 12               | (12 ≤ 20), verdadero | 18+12, 30                | 15      |
| 6         | 15               | (15 ≤ 20), verdadero | 30+15, 45                | 18      |
| 7         | 18               | (18 ≤ 20), verdadero | 45+18, 63                | 21      |
| 8         | 21               | (21 ≤ 20), falso     | Ja hem sortit del bucle  |         |

Antes de dar por cerrado este ejemplo, existe un aspecto bastante significativo que vale la pena tener en cuenta cuando se usan estructuras de repetición. En este caso concreto, qué ocurre si cuando le preguntan el valor hasta dónde se quiere sumar escriba 0 o un número negativo? Bien, para contestar esta pregunta hay que recordar que la condición se evalúa justo antes de cada iteración, incluida la primera vez que se llega a la sentencia `while`. En este caso, tan sólo empezar, la condición lógica no se cumple, por lo que nunca se llega a ejecutar ninguna iteración.

Recuerde, pues, que al usar una sentencia `while` si la primera vez que se evalúa la condición lógica esta evalúa a false el código incluido dentro del bucle no se llegará a ejecutar nunca.

### 3.2.5 Ejemplo: acumular cálculos

Los ejemplos que ha visto hasta ahora tenían en común el uso de una variable de control de tipo contador, a partir de la cual se puede ver de forma más o menos clara cuántas iteraciones realizará el bucle. En todos los casos, la particularidad es que éste contador sirve como valor auxiliar, que incluso puede ayudar a realizar los cálculos que queremos, pero no es propiamente el valor resultante. En el ejemplo de imprimir varios caracteres '-' el resultado es el hecho de imprimir por pantalla. En el de la mesa de multiplicar, el resultado que deseamos es el resultado de la multiplicación. Por último, en el ejemplo de sumar múltiplos de tres, el resultado es la variable misma resultado.

Ahora bien, en una estructura de repetición, la evolución del mismo resultado que se está calculando puede ser la señal de salida para dejar de realizar iteraciones. Éste sería el caso utilizar variables con el papel de acumulador. Un ejemplo de este comportamiento, en el que una variable se va modificando de forma que se deja de iterar cuando ésta contiene el resultado final, es el cálculo de la operación módulo con enteros (%).



Un acumulador refleja las modificaciones en el resultado que acercan el final del bucle. Imagen de Paul Carvill

El módulo calcula el residuo que se obtiene al dividir un número entero (el dividendo) entre otro (el divisor).

Una estrategia simple para calcularlo es ir restando el divisor al dividendo hasta que ya no se puede hacer más, puesto que daría negativo. En este caso, el valor del dividendo se va modificando directamente hasta encontrar la solución.

Si s'estructura pas per pas, seria:

1. Se pregunta cuál es el dividendo.
2. Se lee.
3. Se pregunta cuál es el divisor.
4. Se lee.
5. Si el dividendo es menor que el divisor, ya hemos terminado. El divisor ya es resultado del módulo.
6. De lo contrario, el divisor se queda con el divisor.
7. Se vuelve a comprobar el paso 5.

Aunque se ha usado la palabra “si” en el paso 5, observando atentamente los pasos 5 a 7 se ve que en realidad denotan un bucle, ya que se van repitiendo hasta que se produce una condición concreta: que el valor del divisor es menor que el del dividendo. Por tanto, la condición para seguir repitiendo las instrucciones es que el dividendo sea igual o mayor que el divisor.

El código fuente del programa que realiza estos pasos sería el siguiente. Pruebe que funciona en su entorno de trabajo. Para realizar la ejecución más aclaratoria, en cada iteración se muestra por pantalla cómo se va modificando el dividendo.

```
1 import java.util.Scanner;
2 //Un programa que lee un entero y lo muestra por pantalla.
3 clase pública Módulo {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         //NO 1 i 2
7         System.out.print("¿Cuál es el dividendo? ");
8         int dividendo = lector.nextInt();
9         lector.nextLine();
10        //NO 2 i 3
11        System.out.print("¿Cuál es el divisor? ");
12        int divisor = lector.nextInt();
13        lector.nextLine();
14        //NO 5
15        mientras (dividendo >= divisor) {
16            //NO 6
17            dividendo = dividendo - divisor;
18            System.out.println("Bucle: per ara el dividend val " + dividend + ".");
19            //PAS 7: Simplemente equivale a decir que damos la vuelta al bucle para evaluar la condición
```



```

20     }
21     System.out.println("El resultado final es"           + dividendo + ".");
22 }
23 }

```

La tabla 3.4 muestra la evolución del valor del dividendo para cada iteración para el cálculo de 17%4 (que debe dar 1). El resultado final es el valor acumulado en la variable dividendo cuando sale del bucle.

Tabla 3. 4. Evolución del bucle para calcular el módulo usando un acumulador

| Iteración | Inicio del bucle     | Alambre en bucle    |                         |
|-----------|----------------------|---------------------|-------------------------|
|           | valor de 'dividendo' | Condición vale      | valor de 'dividendo'    |
| 1         | 17                   | (17 > 4), verdadero | 17 - 4 = 13             |
| 2         | 13                   | (13 > 4), verdadero | 13 - 4 = 9              |
| 3         | 9                    | (9 > 4), verdadero  | 9 - 4 = 5               |
| 4         | 5                    | (5 > 4), verdadero  | 5 - 4 = 1               |
| 5         | 1                    | (1 > 20), falso     | Ja hem sortit del bucle |

Reto 3: el uso de contadores y acumuladores no es excluyente, sino que puede ser complementario. Piense cómo se podría modificar el programa para calcular el resultado del módulo y la división entera a la vez. Recuerde que la división entera simplemente sería contar cuántas veces se ha podido restar el divisor.

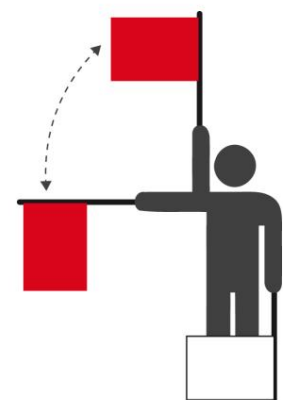
### 3.2.6 Ejemplo: semáforos

El último caso que queda por estudiar es el uso de un semáforo para indicar de forma explícita si ya no es necesario realizar iteraciones. Esta estrategia de uso de variables de control se basa en situaciones en las que decidir si se continúa iterando un bucle. no se puede predecir o calcular de acuerdo a un valor que va aumentando o disminuyendo. Simplemente, es necesario ir repitiendo hasta que se cumpla una condición muy concreta. Entonces, de lo que se dispone es de una variable de control, normalmente de tipos booleano, sobre la que se hace una asignación explícita que provocará que la condición lógica evalúe directamente a false y se salga del bucle.

Un ejemplo de este caso es un programa en el que es necesario adivinar un valor secreto. Mediante una estructura de selección es posible establecer si se ha acertado o no, pero lo que no tiene sentido es que cada vez que se quiera intentar adivinarlo, se haya que ejecutar el programa de nuevo. Lo normal es que se pregunte al usuario hasta que acierte. En este caso, sin embargo, no hay un valor que poco a poco, paulatinamente, va variando hasta que podamos establecer que debemos dejar de iterar. Sucede repentinamente para continuar preguntando a que ya no sea necesario, según la condición de si se ha acertado o no. Ésta puede darse en cualquier iteración, pero es imposible amar cuando, ya que depende totalmente del valor introducido por el usuario.

Si describimos lo que hay que dar paso a paso, sería:

1. Decidir cuál será el número para adivinar.



Un semáforo o banderola avisa:  
"¡Objetivo cumplido! No hace falta iterar más."

2. Pedir que se introduzca un número por el teclado.
3. Llegir-lo.
4. Si el número no es el valor secreto:
  - (a) Hay que avisar de que se ha fallado.
  - (b) Volver al paso 2.
5. De lo contrario, ya hemos terminado. Mostrar una felicitación.

Los pasos 2 a 4 denotan que existe una estructura de repetición, la condición de la cual es que hay que iterar mientras el valor secreto no acierte.

El código correspondiente a este proceso es el que se muestra a continuación. Pruebe que, efectivamente, sigue preguntando números hasta que se acierta el valor secreto. En este caso, la variable de control que hace de semáforo es `haEncertat`. Fijese que, evidentemente, es necesario que el valor inicial sea el que permitirá que, como mínimo, haya una iteración del bucle (la condición evalúe a `true`). De lo contrario, nunca s'entraría al bucle.

```

1 import java.util.Scanner;
2 //Un programa en el que es necesario adivinar un número.
3 public class Adivina {
4     //NO 1
5     public static final int VALOR_SECRET = 4;
6     public static void main (String[] args) {
7         Scanner lector = new Scanner(System.in);
8         System.out.println("Empezamos el juego.");
9         System.out.println("Endevina el valor enter, entre 0 i 10.");
10        booleano haEncertat = falso;
11        mientras (!haEncertat) {
12            System.out.print("¿Qué valor crees que es? ");
13            int uservalue = reader.nextInt();
14            lector.nextLine();
15            Si (SECRET_VALUE != uservalue) {
16                System.out.print("¡Has fallado! Vuelve a intentarlo...");
17            } demás {
18                haEncertat = verdadero;
19            }
20        }
21        System.out.println("Enhorabuena. ¡Por fin has acertado!");
22    }
23 }

```

La tabla 3.5 muestra la evolución de las iteraciones del bucle cuando alguien prueba números 3, 6, 9 y 4, en ese orden.

Tabla 3. 5. Evolución del bucle para adivinar un valor secreto

| Iteración | Inicio del bucle     | Alambre en bucle   |                         |                      |
|-----------|----------------------|--------------------|-------------------------|----------------------|
|           | opción 'hasEncertat' | Condición vale     | Valor 'uservalue'       | opción 'hasEncertat' |
| 1         | FALSO                | (falso), verdadero | 3                       | FALSO                |
| 2         | FALSO                | (falso), verdadero | 6                       | FALSO                |
| 3         | FALSO                | (falso), verdadero | 9                       | FALSO                |
| 4         | FALSO                | (falso), verdadero | 4                       | verdadero            |
| 5         | verdadero            | (verdadero), falso | Ja hem sortit del bucle |                      |

### 3.2.7 Ejemplo: semáforos y contadores a la vez

La categorización de las variables de control en tres modelos distintos no significa que una estructura de repetición siempre deba depender de una única variable.

Recuerde que la condición lógica se representa como una expresión booleana y, por tanto, puede ser tan compleja como se quiera. En algunos casos, puede haber más de una condición bajo la cual se considera que no es necesario realizar más iteraciones de un bucle.

Vuelva a echar un vistazo al programa para adivinar un valor secreto. Ahora ya tiene algo más de sentido, pero todavía hay un aspecto que quizás le hace un poco extraño: el programa no se acabará hasta que adivine el valor secreto. Continuará preguntando una y otra vez hasta que acierte. Si no acierta nunca, nunca se acabará.

Quizás sería más razonable poner un límite al número de intentos, por lo que si se falla más de un cierto número de veces se considera que ha perdido el juego y se termina el programa. En este caso, la condición lógica debe prever dos posibilidades: si se ha acertado el valor secreto o se ha agotado el número de intentos. Es decir, combinar un semáforo y un contador.

---

Nunca habrá más iteraciones que el valor estipulado en el contador de intentos, de acuerdo con su incremento.

---

Descrito paso por paso, podría ser:

1. Decidir cuál será el número a adivinar y el máximo de intentos.
2. Pedir que se introduzca un número por el teclado.
3. Llegir-lo.
4. Descontar un intento.
5. Si el número no es el valor secreto, y si no se han agotado los intentos:
  - (a) Hay que avisar de que se ha fallado.
  - (b) Volver al paso 2.
6. De lo contrario, ya hemos terminado.
7. Si el último número dicho es el valor secreto, debe decirse que se ha ganado.
8. Si el último número dicho no es el valor secreto, debe decirse que se ha perdido.

La característica más importante de este problema, en la que hay más de una posibilidad bajo la cual el bucle deja de iterar, es, una vez se sale, detectar el motivo exacto por el que ha salido adelante. Antes, salir del bucle siempre implicaba que había acertado el valor secreto, pero ahora ya no. Éste es el papel que tienen los pasos 7 y 8. En este caso, una manera de ver si las iteraciones han terminado porque se ha adivinado el valor secreto o porque se han agotado los intentos es mirando el último valor que se ha entrado.

El programa que llevaría a cabo este juego sería el siguiente. Intente ponerlo en marcha.

```
1 import java.util.Scanner;
2 //Un programa en el que es necesario adivinar un número.
3 public class AdivinaSemafor {
4     //NO 1
5     public static final int VALOR_SECRET = 4;
6     public static final int MAX_INTENTS = 3;
7     public static void main (String[] args) {
8         Scanner lector = new Scanner(System.in);
9         System.out.println("Empezamos el juego.");
10        System.out.println("Adivina el valor entero, entre 0 y 10, en tres intentos."
11        );
12        booleano haEncertat = falso;
13        entero intenciones = MÁXIMO_INTENTOS;
14        //Para garantizar el ámbito correcto de la variable, ahora es necesario declararla.
15        entero valorUsuario = 0;
16        mientras ((!haEncertat)&&(intents > 0)) {
17            //NO 2 i 3
18            System.out.print("¿Qué valor crees que es? ");
19            Valor del usuario = lector.nextInt();
20            lector.nextLine();
21            //NO 4
22            intenciones = intenciones - 1;
23            //NO 5 i 6
24            Si (SECRET_VALUE != uservalue) {
25                System.out.print("¡Has fallado! Vuelve a intentarlo.");
26            } demás {
27                haEncertat = verdadero;
28            }
29        }
30        si (UserValue == SECRET_VALUE) {
31            //PAS 7: Se ha salido del bucle porque el valor era correcto.
32            System.out.println("Enhorabona. Has encertat!");
33        } demás {
34            //PAS 8: Se ha salido del bucle porque se han agotado los intentos.
35            System.out.println("Intentos agotados. ¡Has perdido!");
36        }
37 }
```

Antes de seguir, vale la pena subrayar que, en este programa, el valor leído desde el teclado se usa fuera del bucle, en la sentencia yf/los. Por tanto, para garantizar que el ámbito de la variable valorUsuario es el correcto, debe declararse con anterioridad y dentro del mismo blog. Si se declara dentro del código del bucle, habrá un error.

La tabla 3.6 muestra un ejemplo de evolución del bucle para un caso en el que se intentan los valores 2 y 4, por lo que se acierta el valor antes de agotar los intentos. En la última fila se subraya la parte de la condición que es determinante para que evalúe a falso. Nótese que, al salir del bucle, valor Usuario vale 4, razón por la cual la estructura de selección considerará que se ha ganado.

Tabla 3. 6. Evolución del bucle para adivinar un valor secreto, con intentos limitados, ganando

| Iteración | Inicio del bucle    |                        |                               | Alambre en bucle           |                     |                     |
|-----------|---------------------|------------------------|-------------------------------|----------------------------|---------------------|---------------------|
|           | 'haEncertat'<br>val | 'intentos' vale<br>val | Condición<br>val              | 'Valor del usuario'<br>val | 'haEncertat'<br>val | valor de 'intentos' |
| 1         | FALSO               | 3                      | (!falso)&&(3>0),<br>verdadero | 2 falso                    |                     | 2                   |
| 2         | FALSO               | 2                      | (!falso)&&(2>0),<br>verdadero | 4 verdadero                |                     | 1                   |
| 3         | verdadero           | 1                      | (!verdadero)&&(1>0),<br>FALSO | Ja hem sortit del bucle    |                     |                     |

También es interesante ver qué ocurre si se fallan todos los intentos, por ejemplo introduciendo 2, 5 y 7. Esto se ve en la tabla 3.7. En este caso, al salir del bucle, valorUsuario vale 7, por lo que la estructura de selección considerará que se ha perdido.

Tabla 3. 7. Evolución del bucle para adivinar un valor secreto, con intentos limitados, perdiendo

| Iteración | Inicio del bucle    |                 | Alambre en bucle              |                            |                     |                     |
|-----------|---------------------|-----------------|-------------------------------|----------------------------|---------------------|---------------------|
|           | 'haEncertat'<br>val | 'intentos' vale | Condición<br>val              | 'Valor del usuario'<br>val | 'haEncertat'<br>val | valor de 'intentos' |
| 1         | FALSO               | 3               | (!falso)&&(3>0),<br>verdadero | 2 falso                    |                     | 2                   |
| 2         | FALSO               | 2               | (!falso)&&(2>0),<br>verdadero | 5 falso                    |                     | 1                   |
| 3         | FALSO               | 1               | (!falso)&&(1>0),<br>verdadero | 7 falso                    |                     | 0                   |
| 4         | FALSO               | 0               | (!falso)&&(0>0),<br>FALSO     | Ja hem sortit del bucle    |                     |                     |

Por último, por la complejidad del ejemplo, también vale la pena estudiar con detalle un caso especial: cuando se acierta en el último intento, por ejemplo, al introducir sucesivamente los valores 2, 5 y 4. Este caso es especial, ya que la condición lógica evalúa a false como resultado de que las dos expresiones individuales también evalúan a false, tanto la que comprueba si se han agotado el número de intentos como la que comprueba si ha acertado. Ahora bien, dado que el último valor que se ha intentado es lo correcto, la estructura de selección considera que se gana el juego. Esto se puede ver más detalladamente en la tabla 3.8.

Tabla 3. 8. Evolución del bucle para adivinar un valor secreto, con intentos limitados, ganando in extremis

| Iteración | Inicio del bucle    |                 | Alambre en bucle              |                            |                     |                     |
|-----------|---------------------|-----------------|-------------------------------|----------------------------|---------------------|---------------------|
|           | 'haEncertat'<br>val | 'intentos' vale | Condición<br>val              | 'Valor del usuario'<br>val | 'haEncertat'<br>val | valor de 'intentos' |
| 1         | FALSO               | 3               | (!falso)&&(3>0),<br>verdadero | 2 falso                    |                     | 2                   |
| 2         | FALSO               | 2               | (!falso)&&(2>0),<br>verdadero | 5 falso                    |                     | 1                   |
| 3         | FALSO               | 1               | (!falso)&&(1>0),<br>verdadero | 4 verdadero                |                     | 0                   |
| 4         | verdadero           | 0               | (!verdadero)&&(0>0),<br>FALSO | Ja hem sortit del bucle    |                     |                     |

### 3.3 Repetir al menos una vez: la sentencia "do/while"

Aunque la sentencia while es más que suficiente para llevar a cabo prácticamente cualquier estructura de repetición, hay otras que se adaptan mejor a ciertos casos muy concretos. Su aportación consiste exclusivamente en facilitar la lectura del código o permitir llevar a cabo algunas tareas de forma automática.

La sentencia do/while permite repetir la ejecución del bucle mientras se verifique la condición lógica. A diferencia de la sentencia while, la condición se verifica al término de cada iteración. Por tanto, independientemente de cómo evalúe la condición, como mínimo siempre se llevará a cabo la primera iteración.

Al contrario que la sentencia while, la sentencia do/while no es tan común en los lenguajes de programación.

### 3.3.1 Sintaxis y estructura

Para realizar este tipo de control sobre las iteraciones de un bucle, la sintaxis de esta sentencia en lenguaje Java es la siguiente:

```
1 hacer {  
2     Instrucciones para ejecutar dentro del bucle  
3 } while (expresión booleana);
```

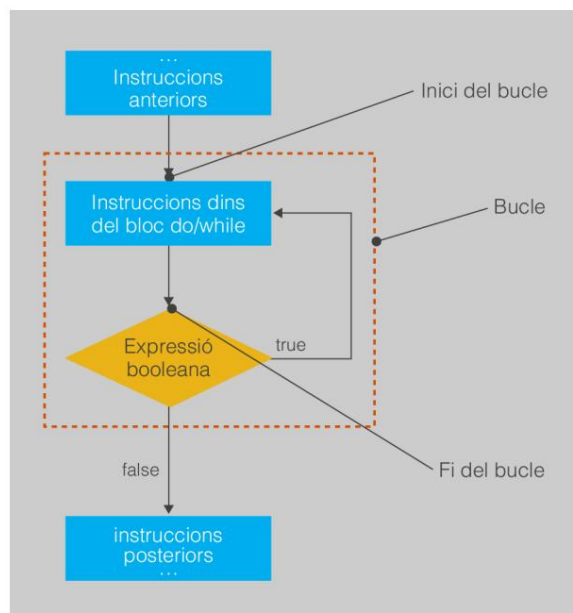
Como puede ver, es muy parecido a la sentencia while, pero invirtiendo el lugar donde aparece la condición lógica. Para poder distinguir claramente dónde comienza el bucle se usa la palabra clave do. Las instrucciones del bucle siguen rodeadas entre claves, {...}.

La figura 3.3 muestra un diagrama del flujo de control de esta sentencia y establece también el orden bajo el cual se evalúa la expresión que representa la condición lógica, en este caso al final, con vistas a decidir si es necesario ejecutar o no una nueva iteración del bucle.

hacer/mientras

El nombre de la sentencia do/while significa: "Haz esto, y sigue haciéndolo mientras se cumpla esta condición".

Figura 3.3. Diagrama de flujo de control de una sentencia "do-while"



### 3.3.2 Ejemplo: control de entrada por teclado

En un ejemplo anterior se ha visto ya la utilidad de las estructuras de repetición a la hora de preguntar datos al usuario. Ahora bien, un ámbito en el que todavía es más útil y más frecuente usarlo es el de controlar si el dato se corresponde con alguno de los valores esperados antes de darlo por bueno y usarlo. Con lo que ha visto hasta el momento, si el usuario se equivoca y escribe algún dato erróneo (por ejemplo, fuera del rango esperado), sois capaces de detectarlo y mediante una estructura de selección avisar al usuario, pero entonces el programa termina. Esto no tiene mucho sentido; sería mucho más cómodo simplemente volver a preguntarle repetidamente hasta que introduzca un valor que se adecue a los valores esperados.

Aunque con un uso sensato de la sentencia while se puede llevar a cabo esta tarea, reemplazarla por la sentencia do/while puede ser especialmente cómodo. El motivo principal es que en estos casos, antes de evaluar si el dato es correcto o no, primero es necesario haberlo leído. Por tanto, es más intuitivo evaluar la condición lógica al final y no al principio del bucle, una vez realmente ya se dispone del dato.

Normalmente, al usar esta sentencia, la condición lógica evalúa directamente el valor a controlar, para ver si cumple los requisitos establecidos. El propio dato leído hace las funciones de semáforo.

El esquema general para ello sería:

1. Pedir el dato al usuario.
2. Leerla.
3. Mirar si el dato es correcto. Si no lo es, volver al punto 1.
4. Si es correcta, ya podemos operar con ella.

Los pasos 1 a 3 identifican que existe un bucle que tiene como condición lógica que los datos escritos no son correctos. Ahora bien, la evaluación de este hecho se realiza al final, no al principio de la secuencia de instrucciones. Además, se cumple que, incluso en el mejor caso, si el usuario no se equivoca, los pasos 1 a 3 siempre se realizarán al menos una vez.

Por ejemplo, pruebe el siguiente programa, que garantiza que cualquier valor introducido estará entre 0 y 10 (como es el caso de lo que se espera en el ejemplo anterior de adivinar un número).

```
1 importe java.util.Scanner; 2 //Vamos a leer un entero entre
0 y 10. 3 public class ValorFitat { public static void main (String[] args) { Scanner lector =
new Scanner(System.in); int valorUsuario = 0; do { //PAS 1 y
4
2: Al menos, seguro que se pregunta una vez.
5
6
7
8
9
10
System.out.print("Ingrese un número entero entre 0 y 10: "); valueUser = reader.nextInt();
```

```

11     lector.nextLine();
12     //PAS 3: Sólo tiene sentido evaluar si 'valor' es válido una vez se ha leído.
13     } mientras ((valorUsuario < 0)|| (valorUsuario > 10));
14     //NO 4: Totalmente correcto
15     System.out.println("Dada correcta. Has escrit " + valorUsuario);
16     }
17 }

```

La tabla 3.9 muestra un ejemplo de evolución del bucle con la entrada de los números -4, 15 y 4. En este caso, la columna de evaluación de la condición se encuentra al final del bucle. En la evaluación, se subraya la parte de la expresión que es determinante porque evalúe a true y se provoque una nueva iteración.

Tabla 3. 9. Evolución del bucle para garantizar un valor entre 0 y 10 con una sentencia do/while

| Iteración | Inicio del bucle  | Alambre en bucle        |                                       |
|-----------|-------------------|-------------------------|---------------------------------------|
|           | Valor 'uservalue' | Valor 'uservalue'       | Condición vale                        |
| 1         | 0                 | -4                      | $(-4 < 0) \vee (-4 > 10)$ , verdadero |
| 2         | -4                | 15                      | $(15 < 0) \vee (15 > 10)$ , verdadero |
| 3         | 15                | 4                       | $(4 < 0) \vee (4 > 10)$ , falso       |
| 4         | 4                 | Ja hem sortit del bucle |                                       |

En realidad, en este caso, una vez la condición ha evaluado a false en la tercera iteración ya se sigue directamente con la siguiente instrucción del programa. No hay ningún nuevo salto al inicio del bucle, al contrario de lo que ocurre con la sentencia while. La evaluación de la condición se realiza al final de cada iteración, y no antes de empezar cada una.

Reto 4: aplique esta comprobación a alguno de los ejemplos anteriores vistos hasta el momento. Por ejemplo, otros casos en los que tiene sentido comprobar si un valor pertenece a un rango esperado para garantizar que un precio es positivo, o si un número de mas está entre 1 y 12.

### 3.4 Repetir un cierto número de veces: la sentencia "for"

En algunas situaciones especiales ya se conoce, a priori, la cantidad exacta de veces que habrá que repetir el bucle. En tal caso es útil disponer de un mecanismo que represente de forma más clara la declaración de una variable de control de tipos contador, la especificación de hasta dónde se debe contar, y que al final de cada iteración incremente o disminuya su valor de forma automática, en lugar de tener que hacerlo nosotros. Automatizar este último punto es muy importante, ya que evita que por un olvido no se haga y se acabe generando un bucle infinito.

Olvidarse de modificar una variable contador es un error muy típico cuando se usan estructuras de repetición.

La sentencia for permite repetir un número determinado a veces un conjunto de instrucciones.

La mayoría de lenguajes suelen disponer del equivalente a esta sentencia.



### 3.4.1 Sintaxis y estructura

La sintaxis de esta sentencia en lenguaje Java es algo más compleja, ya que intervienen muchos factores. Hay que especificar tres apartados especiales separados por punto y coma (;):

```
1 for (inicialización contador ; expresión booleana ; incremento contador) {  
2     Instrucciones para ejecutar dentro del bucle  
3 }
```

para

El nombre de la sentencia for significa:  
"Para ver cuántas veces hay que hacer  
esto, cuenta de ese valor a ese  
otro".

La descripción de cada apartado es la siguiente:

- **Inicialización contador:** se trata de la inicialización de una variable de tipo numérico que servirá como contador. Es exactamente igual que asignar un valor a una variable cualquiera (identificador = valorInicial).  
Si se desea, se permite declarar la variable a la vez que se inicializa (tipo identificador = valorInicial).
- **Expresión booleana:** se trata de la condición lógica que indica si es necesario realizar una nueva iteración o no, al igual que en el resto de estructuras de repetición.
- **Incremento:** se trata de una instrucción que modifica el valor del contador, normalmente una asignación. Esta instrucción se ejecuta automáticamente al final de cada iteración. A pesar de su nombre, puede ser tanto un incremento como una disminución del valor.

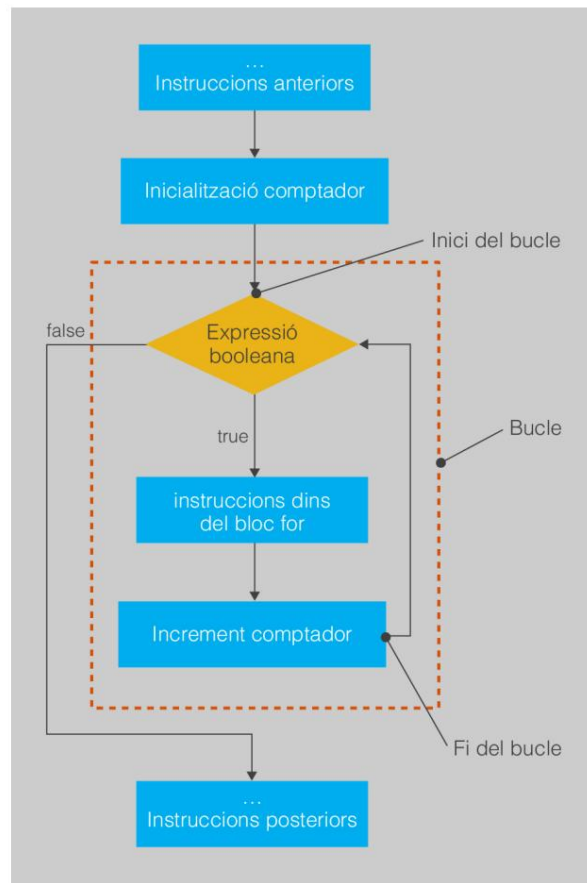
La figura 3.4 muestra un diagrama de flujo de control de esta sentencia e indica cuándo se ejecuta cada elemento de la declaración de la sentencia y cuándo se evalúa la expresión que denota la condición lógica. Nótese que la inicialización sólo se hace una vez, justo antes de realizar la primera evaluación de la condición lógica.

En cambio, el incremento del contador se realiza al final de cada iteración, como si fuera la última instrucción escrita dentro del bucle.

De hecho, la sentencia for es equivalente a una sentencia while que siga la siguiente estructura:

```
1 inicialización contador 2 while (expresión  
booleana) {  
3     Instrucciones para ejecutar dentro del bucle  
4     Incremento contador  
5 }
```

Figura 3. 4. Diagrama de flujo de control de una sentencia "for"



### 3.4.2 Ejemplo: la tabla de multiplicar, versión 2

La sentencia `for` se usa normalmente en lugar de `while` en casos en los que es necesaria una variable de control que va variando su valor en una cantidad fija a cada iteración, ya sea incrementándose o decreciente. Es lo que ocurre, por ejemplo, con los casos controlados por un contador, como los primeros ejemplos de la sentencia `while`.

#### El operador postincremento/decremento

Por poner un ejemplo de la sentencia `for`, se introducirán unos nuevos operadores unarios existentes en algunos lenguajes, como Java. Éstos son especialmente útiles para especificar el autoincremento en el tercer apartado de la sentencia `for`. Se trata del postincremento (`++`) y del postdecremento (`--`). El resultado de aplicarlo es que se suma o resta 1 al valor de la variable, respectivamente. La sintaxis es: `identificadorVariable++` e `identificadorVariable--`, en cada caso. Por ejemplo, `i++`, `valor--`, etc.

Su particularidad es que, como operador, puede utilizarse directamente dentro de una expresión. En ese caso, a la hora de evaluarla su precedencia es la primera.

A continuación se encuentra la adaptación a la sentencia `for` del ejemplo de la tabla de multiplicar. Compruebe que hace exactamente lo mismo.

```
1 import java.util.Scanner;
2 //Un programa que muestra la taula de multiplicar d'un nombre, usant 'for'.
3 classe pública TaulaMultiplicarFor {
4     public static void main (String[] args) {
5         //S'inicialitza la biblioteca.
6         Scanner lector = new Scanner(System.in);
7         //Pregunta el nombre.
8         System.out.print("¿Qué tabla de multiplicar quieres? ");
9         int tabla = lector.nextInt();
10        lector.nextLine();
11        //El contador servirá para realizar cálculos.
12        //En lugar de 'i++' también se podría escribir 'i = i + 1'.
13        para (int i = 0; i <= 10; i++) {
14            int resultado = tabla*i;
15            System.out.println(taula + " * " + i + " = " + resultado);
16        }
17        System.out.println("Esta ha sido la tabla del " + tabla + " + mesa);
18    }
19 }
```

De hecho, la tabla de evolución del bucle es exactamente igual que la de la versión original con la sentencia while (tabla [3.2](#)).

Reto 5: como ocurre con los programas basados en la sentencia while con una variable de control de tipo contador, el contador de una sentencia for no necesariamente debe modificarse sumando/restando de uno en uno. Adapte el ejemplo de la suma de múltiplos de tres usando la sentencia for.

### 3.4.3 Ejemplo: más allá de sumar y restar

De la misma manera que nada obliga a que un contador se incremente de uno en uno, tampoco es imprescindible que la operación sea una suma o una resta. Cualquiera operación que garantice que el valor se va acercando a un caso en el que la condición lógica evaluará a false y se saldrá del bucle es suficiente. Esto también es cierto para en el apartado de autoincremento de la sentencia for.

Por ejemplo, el código siguiente utiliza una sentencia for para mostrar todas las potencias de dos menores o iguales que un hito establecido por el usuario. Para ello, incrementa la variable de control (en este caso, con el papel de acumulador) haciendo multiplicaciones. Compílelo y ejecútelo para ver que, efectivamente, es así.

```
1 import java.util.Scanner;
2 //Vamos a calcular potencias de dos.
3 classe pública Potencias {
4     public static void main (String[] args) {
5         Scanner lector = new Scanner(System.in);
6         System.out.print("¿Hasta qué valor quieres ir buscando potencias de dos? ");
7         int valor = lector.nextInt();
8         lector.nextLine();
9         //La variable "i" se incrementa multiplicando en lugar de sumando.
10        //"" es el mismo resultado.
11        para (int i = 1; i <= valor; i = i*2) {
12            System.out.println(i);
13        }
14    }
15 }
```

La tabla 3.10 muestra la evolución del bucle para poner como límite el valor 70. Recuerde que el valor inicial de la variable y en la primera iteración del bucle se debe a la inicialización dentro del primer apartado de la sentencia for, mientras que el valor al final se debe al autoincremento especificado en el tercer apartado ( $i = 2 * i$ ).

Tabla 3. 1 0. Evolución del bucle para calcular potencias de 2 usando una sentencia "for"

| Iteración | Inicio del bucle |                      | Alambre en bucle        |
|-----------|------------------|----------------------|-------------------------|
|           | 'i' val          | Condición vale       | 'i' val                 |
| 1         | 1                | (1 ≤ 70), verdadero  | 2 * 1 = 2               |
| 2         | 2                | (2 ≤ 70), verdadero  | 2 * 2 = 4               |
| 3         | 4                | (4 ≤ 70), verdadero  | 2 * 4 = 8               |
| 4         | 8                | (8 ≤ 70), verdadero  | 2 * 8 = 16              |
| 5         | 16               | (16 ≤ 70), verdadero | 2 * 16 = 32             |
| 6         | 32               | (32 ≤ 70), verdadero | 2 * 32 = 64             |
| 7         | 64               | (64 ≤ 70), verdadero | 2 * 64 = 128            |
| 8         | 128              | (128 ≤ 100), falso   | Ja hem sortit del bucle |

3.5 Combinación de estructuras de selección



Hacer bucles dentro de otros bucles es como una "rotonda mágica" Imagen de Wikimedia Commons

Como ocurría con las estructuras de selección, nada impide combinar diferentes estructuras de repetición, imbricadas unas dentro de otras, para llevar a cabo tareas más complejas. En última instancia, la combinación e imbricación de estructuras de selección y repetición conformará su programa.

Cuando se combinan estructuras de repetición es necesario ser muy cuidadosos y tener presente qué variables de control están asociadas a la condición lógica de cada bucle. Tampoco se olvide de sangrar correctamente cada bloque de instrucciones, para poder así identificar rápidamente dónde termina y comienza cada estructura.

3.5.1 Ejemplo: la tabla de multiplicar, versión 3

Normalmente, la combinación de estructuras de repetición siempre está fundamentada por la necesidad de hacer varias veces una tarea que ya de por sí requiere una estructura de repetición. Por ejemplo, suponga que en lugar de querer un programa que muestre una única tabla de multiplicar, como en un ejemplo anterior, se quieren mostrar varias tablas de multiplicar. Desde la del 1 hasta el valor que elija. Hay dos tareas repetitivas: es necesario repetir N veces una tarea que muestra una tabla completa, que es a la vez la repetición de 10 multiplicaciones.

El código que llevaría a cabo esta tarea sería el siguiente. Al tratarse de una tarea basada en un contador, se usará la sentencia for para ambas estructuras repetitivas.

```
1 import java.util.Scanner;
2 //Un programa que muestra N tablas de multiplicar.
3 clase pública TaulaMultiplicarN {
4
5     public static void main (String[] args) {
6         Scanner lector = new Scanner(System.in);
7         System.out.print("¿Hasta qué tabla de multiplicar quieres conocer? ");
8         int tablas = lector.nextInt();
9         lector.nextLine();
10        //Bucle de primer nivel: inicio del bucle que genera N tablas de
11        multiplicar.
12        for(int i = 1; y <= tablas; i++) {
13            System.out.println("La taula del " + i);
14            //Bucle de segundo nivel: inicio del bucle que genera una tabla concreta.
15            para(int j = 1; j <= 10; j++) {
16                entero resultado = i*j;
17                System.out.println(i + " * " + j + " = " + resultado);
18            }
19            //Fi del bucle que genera una taula concreta.
20            System.out.println("-----");
21        }
22        //Fin del bucle que genera N tablas de multiplicar.
23    }
```

Este ejemplo también es bastante interesante, ya que tiene en cuenta el ámbito de las variables que se utilizan como contador para llevar a cabo su tarea. Así pues, la variable *y*, el contador del bucle de primer nivel (el más “externo”), puede ser usada dentro del bucle de segundo nivel (el más “interno”), puesto que se considera declarada hasta la llave que cierra su bloque de código. El caso inverso para *j* no es cierto y hacerlo produciría un error de sintaxis.

### 3.5.2 Ejemplo: adivinar el número secreto, versión 3

Merece la pena mostrar como ejemplo un programa algo más complejo, también basado en la combinación de estructuras de repetición. En un ejemplo anterior se ha visto que este tipo de estructuras son útiles para controlar si la entrada de datos es correcta. Si se combina este ejemplo con el programa para adivinar un número secreto, en el que se pregunta repetidas veces al usuario qué respuesta quiere dar, entonces tendrá el esquema del programa siguiente, que depende de dos estructuras de repetición, una dentro de la otra:

1. Decidir cuál será el número a adivinar y el máximo de intentos.
2. Pedir que se introduzca un número por el teclado.
3. Llegir-lo.
4. Mirar si el valor es correcto. Si no lo es, volver al paso 2.
5. Si es correcto, ya se puede operar.
6. Descontar un intento.
7. Si el número no es el valor secreto y no se han agotado los intentos:

(a) Hay que avisar de que se ha fallado.

(b) Volver al paso 2.

8. De lo contrario, ya hemos terminado.

9. Si el último número dicho es el valor secreto, debe decirse que se ha ganado.

10. Si el último número dicho no es el valor secreto, debe decirse que se ha perdido.

El código sería el que se muestra a continuación. A pesar de la aparente complejidad, fíjese que, básicamente, se parte del programa original de adivinar el valor secreto y que, en la parte donde se pedía el valor, se inserta directamente la estructura de El ejemplo de cómo comprobar que los datos son correctos. Compílelo y ejecútelo para comprobarlo que funciona.

```
1 import java.util.Scanner;
2 //Un programa en el que es necesario adivinar un número.
3 public class AdivinaN {
4     //NO 1
5     public static final int VALOR_SECRET = 4;
6     public static final int MAX_INTENTS = 3;
7     public static void main (String[] args) {
8         Scanner lector = new Scanner(System.in);
9         System.out.println("Empezamos el juego.");
10        System.out.println("Adivina el valor entero entre 0 y 10 en tres intentos.")
11        ;
12        boolean haEncertat = falso;
13        entero intenciones = MÁXIMO_INTENTOS;
14        entero valorUsuario = 0;
15        //Inicio bucle para adivinar el valor.
16        mientras ((!haEncertat)&&(intents > 0)) {
17            //Inicio bucle para obtener un dato desde el teclado.
18            hacer {
19                //PAS 2 y 3: Al menos, seguro que se pregunta una vez.
20                System.out.print("Introduce un valor entero entre 0 y 10: ");
21                Valor del usuario = lector.nextInt();
22                lector.nextLine();
23                //NO 4
24            } mientras ((valorUsuario < 0)|| (valorUsuario > 10));
25            //Fi bucle para obtener un dato desde el teclado.
26            //NO 6
27            intenciones = intenciones - 1;
28            //NO 7 i 8
29            Si (SECRET_VALUE != uservalue) {
30                System.out.print("¡Has fallado! Vuelve a intentarlo.");
31            } demás {
32                haEncertat = verdadero;
33            }
34        }
35        //Fi bucle para adivinar el valor.
36        si (UserValue == SECRET_VALUE) {
37            //PAS 9: Se ha salido del bucle porque el valor era correcto.
38            System.out.println("Enhorabuena. ¡Has acertado!");
39        } demás {
40            //PAS 10: Se ha salido del bucle porque se han agotado los intentos.
41            System.out.println("Intentos agotados. ¡Has perdido!");
42        }
43 }
```

## 3.6 Soluciones de los retos propuestos

### Reto 1:

```
1 import java.util.Scanner;
2 clase pública LiniaPregunta {
3     public static void main (String[] args) {
4         //Parte modificada. Se pregunta el valor del contador.
5         Scanner lector = new Scanner(System.in);
6         System.out.print("¿ Cuántas iteraciones quieres hacer? ");
7         boolean tipusCorrecto = lector.hasNextInt();
8         //Se ha escrito un entero correctamente. Ya se puede leer.
9         si (tipusCorrecto) {
10             //Leemos el número de iteraciones.
11             int iteraciones = lector.nextInt();
12             //Inicializamos un contador.
13             entero i = 0;
14             //¿Ya hemos hecho esto los golpes que toca?
15             mientras (i < iteraciones) {
16                 System.out.print("-");
17                 //Lo hemos hecho una vez, sumamos 1 al contador.
18                 i = i + 1;
19             }
20             //Forzamos un salto de línea.
21             System.out.println();
22         } demás {
23             //No se ha escrito un entero.
24             System.out.println("El valor introducido no es un entero.");
25         }
26         lector.nextLine();
27     }
28 }
```

### Reto 2:

```
1 import java.util.Scanner;
2 clase pública TaulaMultiplicarEnrere {
3     public static void main (String[] args) {
4         //S'inicialitza la biblioteca.
5         Scanner lector = new Scanner(System.in);
6         //Pregunta el nombre.
7         System.out.print("¿Qué tabla de multiplicar quieres ver? ");
8         int tabla = lector.nextInt();
9         lector.nextLine();
10        //El contador servirá para realizar cálculos.
11        entero i = 10;
12        mientras (i >= 1) {
13            int resultado = tabla*i;
14            System.out.println(taula + i = i - 1;      " * " + yo + " = " + resultado);
15        }
16        System.out.println("Esta ha sido la tabla del " + mesa);
17    }
18 }
19 }
```

## Reto 3:

```
1 import java.util.Scanner;
2 clase pública ModulDivisio {
3     public static void main (String[] args) {
4         Scanner lector = new Scanner(System.in);
5         //NO 1 i 2
6         System.out.print("¿Cuál será el dividendo? ");
7         int dividendo = lector.nextInt();
8         lector.nextLine();
9         //NO 2 i 3
10        System.out.print("¿Cuál será el divisor? ");
11        int divisor = lector.nextInt();
12        lector.nextLine();
13        //NO 5
14        int cociente = 0;
15        //NO 6
16        mientras (dividendo >= divisor) {
17            //NO ES 7
18            dividendo = dividendo - divisor;
19            //NO 8
20            cociente++;
21            //PAS 9: Simplemente equivale a decir que damos la vuelta al bucle para evaluar
                la condición
22            System.out.println("Bucle: per ara el dividend val " + dividendo + ".");
23        }
24        //PAS 10: ¡La variable "cociente" tiene el cociente al terminar el bucle!
25        System.out.println("La división es " + cociente + ".");
26        //PAS 11: ¡La variable "dividendo" tiene el módulo al terminar el bucle!
27        System.out.println("El módulo es " + dividendo + ".");
28    }
29 }
```

## Reto 4:

```
1 import java.util.Scanner;
2 clase pública ValorMesFitat {
3     public static void main (String[] args) {
4         Scanner lector = new Scanner(System.in);
5         entero mes = 0;
6         hacer {
7             System.out.print("Introduce un número de mes [1-12]: ");
8             mes = lector.nextInt();
9             lector.nextLine();
10        } mientras ((mes < 1)|| (mes > 12));
11        System.out.println("Dada correcta. Has escrit " + mes);
12        System.out.print("Los días de este mes son...");
13        si (mes == 2) {
14            System.out.println("28 o 29!");
15        } else if ((mes == 4)|| (mes == 6)|| (mes == 9)|| (mes == 11)) {
16            System.out.println("30!");
17        } demás {
18            System.out.println("31!");
19        }
20    }
21 }
```



## Reto 5:

```
1 import java.util.Scanner;
2 class pública SumarMultiplesTresFor {
3     public static void main (String[] args) {
4         Scanner lector = new Scanner(System.in);
5         System.out.print("¿Hasta qué valor quieres sumar múltiplos de 3? ");
6         int límite = lector.nextInt();
7         lector.nextLine();
8         entero resultado = 0;
9         para(int i = 0; i <= límite; i = i + 3 ) {
10             System.out.println("Afejim " + i + "...");
11             resultado = resultado + i;
12         }
13         System.out.println("El resultado final es" + resultado + ".");
14     }
15 }
```